THM | CAMPUS GIESSEN | MNI
TECHNISCHE HOCHSCHULE MITTELHESSEN
Mathematik, Naturwissenschaften und Informatik

A Thesis submitted for the degree of Master of Science

SoC-based Image Classification using Binarized Neural Networks

Bastian Zeller

August 2018

Technische Hochschule Mittelhessen
University of Applied Sciences

Contents

Acknowledgements **xi**

Abstract **xiii**

1 Introduction **1**
1.1 Context . . . 1
1.2 Motivation . . . 1
1.3 Work Assignment . . . 1
1.4 Thesis Structure . . . 2

2 State of Art Review **3**

3 Theoretical Background **7**
3.1 Artificial Neural Networks . . . 7
3.1.1 Perceptron . . . 7
3.1.2 Artificial Neurons . . . 11
3.1.3 Networks of Neurons . . . 13
3.1.4 Learning as a Gradient Descent . . . 17
3.1.5 Backpropagation Algorithm . . . 20
3.1.6 Binarized Neural Networks . . . 24
3.2 Parallel Computing Platforms . . . 28
3.2.1 Overview of Platforms . . . 28
3.2.2 Challanges for Parallel Hardware Architectures . . . 28
3.3 Performance Indicators . . . 30
3.3.1 Classification Accuracy . . . 31
3.3.2 Execution-Time per Frame . . . 31
3.3.3 FPGA Logic Gates used . . . 31
3.3.4 Development Cost . . . 32
3.3.5 Training Cost . . . 32
3.3.6 energy consumption battery life . . . 32

4 System Architecture **33**
4.1 Network Structure . . . 33
4.2 OpenCL Integration . . . 34

5 Implementation Details **37**
5.1 Network Implementation . . . 37
5.1.1 Input Preprocessing . . . 37
5.1.2 Hidden Layers . . . 37
5.1.3 Activation Function . . . 38
5.1.4 Weights Scaling . . . 39
5.1.5 Forward Propagation . . . 40
5.1.6 Gradient Calculation . . . 40
5.1.7 Binarizing Weights . . . 40
5.1.8 Network Optimizations . . . 41
5.2 OpenCL Implementations . . . 46
5.2.1 Matrix Dot Product . . . 46
5.2.2 XNOR Product . . . 48
5.2.3 Compressed XNOR Product . . . 49

6 Analysis **51**
6.1 Performance Measurements . . . 51
6.1.1 Classification Accuracy . . . 51
6.1.2 Execution-Time Per Frame . . . 52
6.1.3 FPGA Logic Gates used . . . 53
6.1.4 Training Cost . . . 54
6.1.5 Development Cost . . . 55
6.2 Evaluation of KPI . . . 55
6.2.1 Classification Accuracy . . . 55
6.2.2 Execution-Time per Frame . . . 56
6.2.3 FPGA Logic Gates used . . . 58
6.2.4 Training Cost . . . 59
6.2.5 Development Cost . . . 59

7 Conclusions **61**
7.1 Summary . . . 61
7.2 Future Work . . . 61

Bibliography **68**

List of Figures

2.1 Advances in ImageNet competition. 4

3.1 A schematic representation of a perceptron 8
3.2 Plot of step function 8
3.3 Parameterization of a perceptron to implement the NAND logic function. 9
3.4 Implementation of a full adder using only NAND logic gates. 10
3.5 Implementation of a full adder with perceptrons. 10
3.6 A schematic representation of an artficial neuron 11
3.7 Plot of sigmoid function 12
3.8 Graph of a feed forward neural network with a single hidden layer. . . 13
3.9 Graph of a recurrent neural network. 14
3.10 Graph of a feed forward network with multiple hidden layers. 15
3.11 Under- and overfitting network models. 16
3.12 Architecture of the LeNet-5. 17
3.13 The surface plane of the Mean Squared Error with 2 inputs 18
3.14 A surface plan of a network showing local and global minima. 18
3.15 Plot of the hyperbolic tangent. 19
3.16 Partial derivative of sigmoid and hyperbolic tangent. 20
3.17 Funtion derivates in network propagation 21
3.18 Funtion derivatives in network propagation 21
3.19 Function composition during network propagation 21
3.20 Function addition during network propagation 22
3.21 Plot of sign function 25
3.22 SIMD versus pipelined architecture. 29
3.23 SIMD versus pipelined branching. 29
3.24 SIMD versus pipelined IO access. 30
3.25 SIMD versus pipelined loop execution. 30

4.1 Graph of the implemented network. 33
4.2 Sequence Diagram for ARM and FPGA cores accessing DDR. 35
4.3 Sequence Diagram for OpenCL Kernels accessing DDR. 36

5.1 Visualization of the dropout technique. 43
5.2 Converging functions that can replace each other. 46

6.1 1 hidden layer kernel timings on ARM and SoC 57
6.2 2 hidden layer kernel timings on ARM and SoC 57
6.3 Comparison of logic utilization for different implementations. 58

List of Tables

3.1 Lookup-Table for the XNOR function. 26

6.1 Classification Accuracy for different network models. 51
6.2 Timings for ANN and XNOR implementations running on ARM core. 52
6.3 Timings for ANN and XNOR implementations accelerated by FPGA. 53
6.4 FPGA Resources allocated compared between different implementations. 53
6.5 Resources allocated in FPGA for different kernels 54
6.6 Training Times for different network implementations. 55
6.7 Training Times for different network implementations. 55
6.8 Speed for matrix operations with different kernels 56
6.9 Resulting lines-of-code in the intermediate code OpenCL generates. . 60

Listings

5.1 Implementation of the activation functions and their derivates 39
5.2 Exaggerating the error of a neurons activation. 45
5.3 Implementation of a matrix dot product in OpenCL. 47
5.4 Implementation of the XNOR product in OpenCL. 48
5.5 Implementation of the compressed XNOR product in OpenCL. 49

Abbreviations in Alphabetical Order

AI: Artificial Intelligence
ANN: Artificial Neural Network
ASIC: Application-specific Integrated Circuit
BNN: Binarized Neural Network
BOVW: Bag-of-Visual-Words
CNN: Convolutional Neural Network
DBN: Deep Belief Network
DSP: Digital Signal Processing
FPGA: Field Programmable Gate Array
GPU: Graphics Processing Unit
LUT: Lookup Table
LOC: Lines of Code
MNIST: Mixed National Institute of Standards and Technology
OpenCL: Open Computing Languare
SGD: Stochastic Gradient Descent
SIFT: Scale-invariant feature transform
SoC: System on Chip
TANH: Hyperbolic Tangent
TPU: Tensor Processing Unit
ReLU: Rectified Linear Unit

Eidesstattliche Erklärung

Hiermit versichere ich, die vorliegende Arbeit selbstständig und unter ausschließlicher Verwendung der angegebenen Literatur und Hilfsmittel erstellt zu haben.

Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Gießen, 12. August 2018

Statutory Declaration

I declare that I have authored this thesis independently, that I have not used other than the declared sources / resources, and that I have explicitly marked all material which has been quoted either literally or by content from the used sources

The Master Thesis was not used in the same or in a similar version to achieve an academic grading or is being published elsewhere.

Gießen, 12. August 2018

Acknowledgements

I would like to express my gratitude to my supervisor, Prof. Dr. Hartmut Weber at THM for the useful comments, remarks and engagement through the learning process of this master thesis. I would also like to acknowledge Prof. Dr. Martin Gräfe at THM as the second reader of this thesis, and I am gratefully indebted to him for his very valuable comments on this thesis.

Finally I must express my gratitude to my loved ones, who have supported me throughout entire process of writing this thesis, both by keeping me harmonious and helping me putting pieces together.

Bastian Zeller

Abstract

TODO: write abstract

keywords: Neural Network, Deep Learning, Embedded Systems, OpenCL, FPGA, SoC, Image Classification,

1 Introduction

This thesis deals with the research and implementation on binarized artificial neural networks used for image classification tasks.

This chapter introduces the context in which neural networks are applied and describes recent challenges with their implementation and optimization.

A thesis on overcoming these challenges is proposed and finally, a roadmap of this thesis will be presented.

1.1 Context

1.2 Motivation

1.3 Work Assignment

This master thesis is related on machine learning, especially image classification by neural networks. Typically, deep neural networks (DNNs) are used for for these tasks. DNNs are computationally expensive and require large memory to store high precision weights.

A new approach for working on image classification tasks with neural networks are Binarized Neural Networks (BNNs) [1, 2, 3]. BNNs only use binary weights and activation which need much less memory than high precision weights for DNNs. So BNNs are suited for an implementation on FPGA hardware.

In this thesis, the classification of handwritten digits or characters from the EMNIST dataset [4] provided by NIST, the National Institute of Standards and Technolgy of US Department of Commerce, has to be used for the evaluation of BNN usage in an SoC device equipped with one Cortex A9 ARM core and an Altera Cyclone IV FPGA. A second task is to compare the utilization of FPGA logic and memory cells for this implementation with other comparable FPGA implementations for image classification [5, 6, 7]

1.4 Thesis Structure

In this master thesis a method to run binarized neural networks on SoC platform consisting ARM processor cores and FPGA logic will be evaluated. The thesis is mainly divided into three parts: theoretical background, methodic and implementation details and the evaluation and analysis the achieved results.

Collecting theoretical background information will help to understand the fundamentals of state-of-the-art technologies available at this time and will enable us to select the best technology suited for the work planned.

- **Chapter 2** analyzes the state of art in image processing and the usage of deep neural networks.
- **Chapter 3** introduces deep learning methods and discusses the advancement of the usage of ANN for image classification tasks Key Performance Indicators are defined that will enable further discussion on the topic.
- In **chapter 4** the system architecture is presented
- **Chapter 5** discusses the implementation of the binarized neural network as well as its representation in the SoC
- In **chapter 6** performance measurements are presented and analyzed according the previously defined Key Performance Indicators
- **Chapter 7** summarizes the key aspects of this thesis, and gives on overview on possible future work building up on this thesis.

2 State of Art Review

The last years have been a renaissance to deep learning. With processing capabilities evolving, the concept developed in the 20th century is gaining more and more interest in the industry. It makes its way into consumer devices like smart phones or tablets [8] and became the basement of home assistance systems [9].

A field nowadays highly dominated by deep learning is image classification, the basement for computer vision. The technique is reaching highest scores in classifying competitions like MNIST [10] or ImageNet [11].

These competitions were formerly approached with conventional image analysis algorithms like Scale-invariant feature transformation (SIFT) [12] or Bag-of-Visual-words (BOVW) [13, p.101 ff.]. These conventional image processing algorithms were providing mitigated results, showing error rates only down to 26.2 percent.

Advances in the ImageNet Large Scale Visual Recognition Challenge were achieved starting in 2012. Authors proposed several architectures based on neural networks that were setting the groundwork on recent image classification research. As visualized in figure 2.1, the error rates have been reduced to outperform even human capabilities:

AlexNet was the first big achievement in image classification. The paper describing the algorithm is one of the most influential papers concerning image classification. It reduced the error rate commonly achieved at this time by 50 percent to an error rate of 15.4%. In 2012 the authors of [14] were the first to use convolutional neural networks on image classification tasks. This approach was ground-breaking an brought the concepts of deep learning into public focus. It was a very basic implementation of 8 layers, using mainly 11x11 filters. Because of the huge computational complexity, AlexNet was run in parallel on 2 GPUs.

While AlexNet focused on implementing only a few layers with rather complex filters, **VGG Net** was implemented by using only 3x3 filters [15]. The authors argue that combining 2 3x3 filters simulate the effect of a larger 5x5 filter, by still providing the computational benefits of a small filter size. They implemented a total of 19 layers which turned out to perform well not only on image classification, but also on localization tasks [15, p. 10]. The model was trained on 4 Nvidia Titan Black GPUs over the period of 3 weeks and could achieve an error rate of 7.3%.

The next advancement in classification tasks was provided by **GoogleNet** in 2014, a project of several Google research groups focusing on self-driving cars, image research and understanding content provided in YouTube videos [16]. It proposed a 22 layer convolutional network which achieved an error rate of 6.7% in the ImageNet competition. The approach is different from all other approaches to this date, because it does not focus on stacking layers on top of each other in a sequential path, but is introducing parallel structures of 1x1, 3x3 and 5x5 filters. The authors state the performance as "trained on a few high-end GPUs within a week".

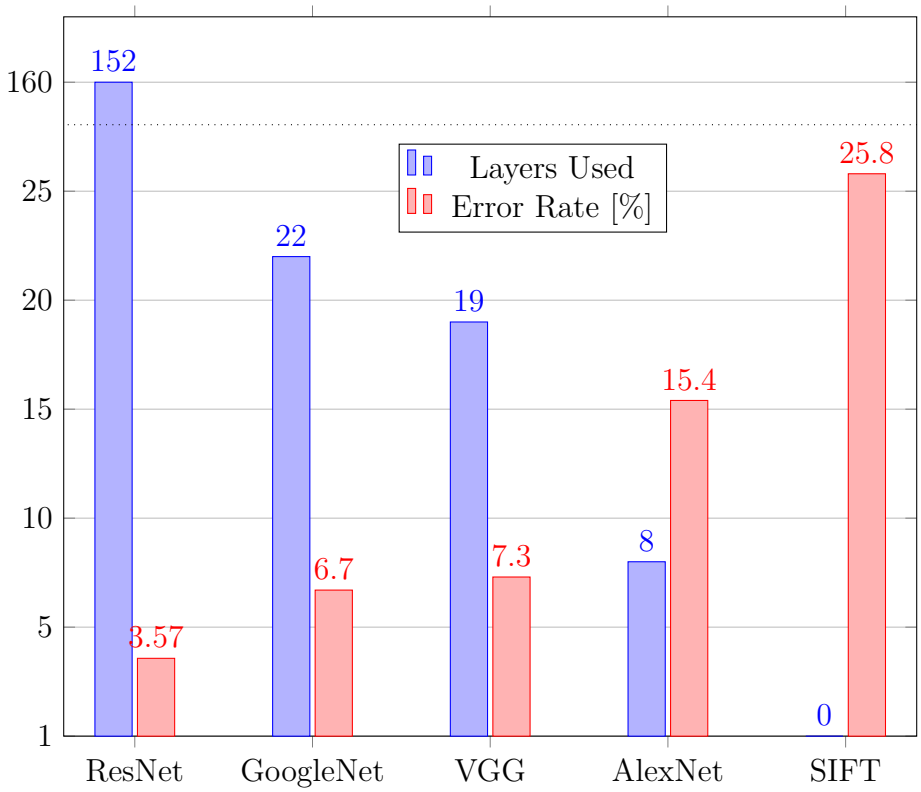


Figure 2.1: Advances in ImageNet competition.

In 2015 the Microsoft Research Asia group proposed the architecture of **ResNet**, a 152-layer network leading to an error rate of 3.6%, which even outperforms human capabilities [17]. The main idea behind ResNet is the "identity shortcut connection", a shortcut path within the network which enables the use of a massive amount of layers, without losing control over it. It was trained on 8 GPUs within 3 weeks.

A side effect on the popularity deep neural networks gained since the proposal of AlexNet in 2012 is the possibility to use them for different tasks besides image classification by using the proposed networks in new ways: Instead of classifying images, researchers could produce new images by mixing attitudes of different images. An

example is the transfer of one artworks style to another image which enables the production of new artworks [18].

All the previously discussed approaches are following the trend of implementing (very) deep convolutional networks. While this leads to higher accuracy, the networks are not the most efficient in terms of size and computational speed. They usually require high-end hardware and very long time to train and execute the network.

In real life applications like the usage in self-driving cars, robotics or in assembly lines, the tasks executed by those networks are supposed to be processed in a very limited time frame and run on limited hardware. With the appearance of deep learning in low power computing devices like smart phones, home assistants and other embedded devices, the demand to reduce the computational complexity of applications implementing deep learning algorithms is even growing faster. This lead researchers to investigate on approaches to minimize the computational cost of those networks and find possibilities to reduce the need for expensive hardware based on multiple high-end GPUs.

While one approach in devices with access to the internet is to outsource complex calculations to cloud service providers [19], research is focusing on approaches to reduce the complexity the calculations of neural networks have by researching on techniques that lower the needs of processing power and memory consumption and thus, are speeding up the run time without sacrificing too much accuracy:

Network pruning is a technique of identifying and removing parts inside neural networks that have the least important influence on the outcome [20]. Methods on identifying those parts can be based on the magnitude of weights and activations or by identifying mutual information within different paths of the network like feature maps that filter similar criteria. The theory of pruning is modeled after the human brains capability to maintain functionality even after damages of big parts of the brain [21, 22].

MobileNets, as proposed by a Google research group, combine recent techniques that allow big convolutional networks being computed on small embedded devices like mobile phones or tablets [23]. Depthwise separable convolutions, previously discovered in the Xception network [24], allow the thinning of convolutional filters. In a 2D environment with multiple channels, e.g. an image with RGB color, a filter is as deep as the number of channels used on the input. In depthwise convolution each channel is presented to a separate filter. This technique is combined with a 1x1 convolution across channels, which allows computation with less parameters as regular convolutional layers have and is thus leading to less operations to compute. This makes the network cheaper and faster. The technique of depthwise separable convolutions is combined with the 2 'hyper parameters' width multiplier and resolution multiplier. The width multiplier is scaling the amount of inputs and outputs for

each layer, while the resolution multiplier is a parameter scaling the size of the input image and the internal representation of every layer, allowing to reduce the size and latency of the network.

The **ShuffleNet** convolutional network "is designed specially for mobile devices with very limited computing power" [25]. By introducing two new operations, pointwise group convolution and channel shuffle, the computational cost of a network can be reduces drastically by trading only a reasonable amount of the networks accuracy. Group convolution is a combination of techniques used in AlexNet and the previously discussed depthwise separable convolution by generalizing them to work on groups of input channels. The channel shuffle operation is extracting information from feature maps and shuffles the among channel groups, leading to increased accuracy and lowered computational speed.

While these implementations focus on implementing the neural networks on mobile processor architectures like the ARM architecture, other researchers focus on limiting and optimizing the complexity of the underlying calculations to allow mapping the networks algorithms on programmable logic like FPGA and ASIC technology.
Binarized neural networks evolved from neural networks with low precision weights [26] and reduce the networks complexity by removing the need for high precision and thus, computational expensive floating-point calculations in processing the networks weights and activations [1]. Weights and activations are represented as binary values, allowing to replace formerly expensive multiplications and trigonometric functions being replace by simple bitwise operations like XNOR or adders.

The usage of FPGA hardware is a reasonable tradeoff between the computational power of GPUs and the flexibility and efficiency of mobile CPU architectures like ARM and can build the basis for specialized machine learning chips. The hardware design synthesized for the FPGA can be converted to an ASIC design efficiently without the need for redesigning the architecture. These custom designs have the advantage of reducing energy consumption drastically. The trend of using FPGA or custom ASIC chips for neural network computation is already applied to big scale data center operations and operators that were previously relying exclusively on CPU-GPU architectures [27] are moving towards FPGA or ASIC architectures: Google is developing custom ASIC accelerators for machine learning tasks using neural networks, called Tensor Processing Units (TPU), which they deploy to data centers [28]. Microsoft and Amazon follow different approaches in their data centers by deploying FPGA based accelerators or SoCs consisting of CPU and FPGA cores to their data centers [29, 30], which are the basis to their cloud services catapult [31] and AWS F1 [30].

3 Theoretical Background

This chapter will introduce the fundamental theory on which later chapters are based. It introduces the theory of neural networks and discusses parallel computing platforms that are suited for neural network computing and image classification. To be able to measure the efficiency of the model developed in this work and compare it to different models, performance measuring units will be defined and discussed.

3.1 Artificial Neural Networks

Artificial neural networks are algorithms loosely modeled after the human brains neuronal structure. They are usually composed out of multiple layers of artificial neurons, a concept this section will introduce. The concept of artificial neurons evolved from the perceptron, a concept introduced in the second half of the twentieth century.

3.1.1 Perceptron

The concept of perceptron was developed in the early 1960's [32, Ch.2, p. 12]. It is a simplified representation of a biological neuron. While modern neural network use more complex models of neurons, the perceptron is introducing the fundamental concepts of nowadays neurons. The purpose of a perceptron is making decisions on problems by weighting evidence.

A perceptron takes several binary inputs $x0, x1, x2, ...$ and produces a single binary output o_j. The inputs are weighted and a sum over all weighted inputs is calculated. This sum is fed into a function called activation function, which produces a binary output [33, pp. 55-75].

The activation function is a step function 3.2 with the zero-point shift shifted by a threshold. It produces an output of 1 for all inputs $x_n \geq \theta_j$ and 0 for all inputs $x_n < \theta_j$. The threshold can also be described as an input parameter of the perceptron

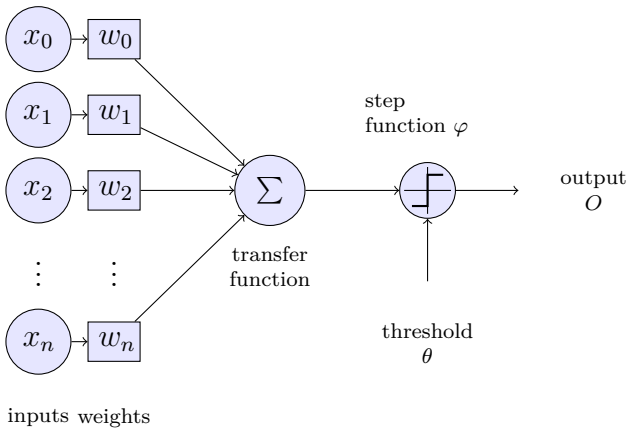


Figure 3.1: A schematic representation of a perceptron

and moved to the other side of the equation. This input parameter is called bias. $b_j \equiv -\theta$. Thus, the activation function can be described with equation 3.1.

$$f(x) = \begin{cases} 0 & \text{if } x + b_j \geq 0 \\ 1 & \text{if } x + b_j < 0 \end{cases} \tag{3.1}$$

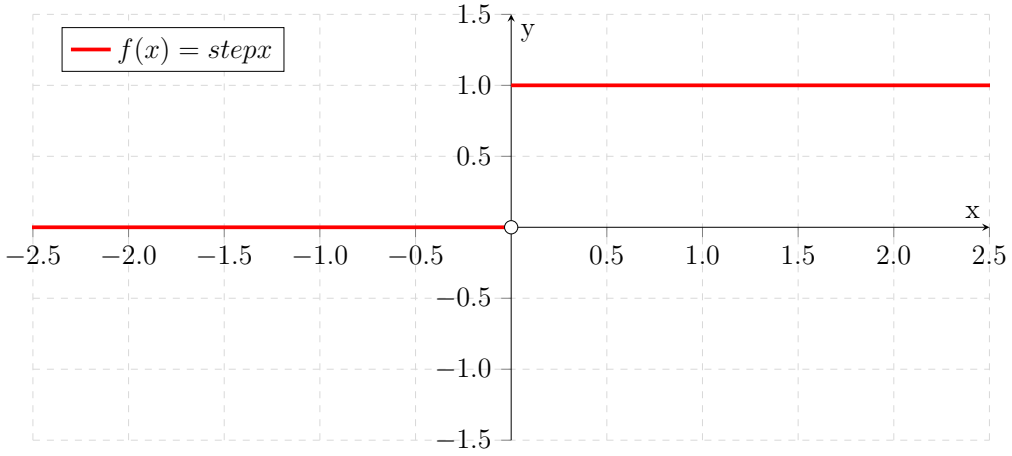


Figure 3.2: Plot of step function

Varying the weights will result in different models of the perceptron and so in different paths of the decision making. Inputs and weights are both vectors of equal length. Thereby the sum of all weighted inputs can be described as the dot product of the input vector x and the weights vector w. Equation 3.2 is showing the relation between

the weighted sum and dot product.

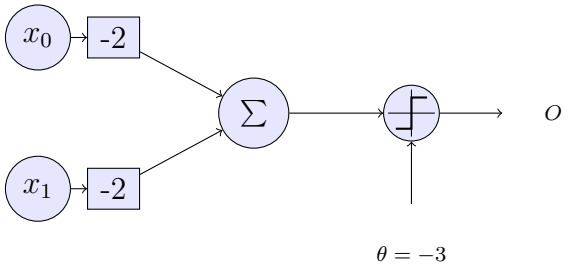
$$w \cdot x \equiv \sum_j w_j * x_j \tag{3.2}$$

The perceptron can be described by the equation 3.3

$$O = \begin{cases} 0 & \text{if } w \cdot x + b \geq 0 \\ 1 & \text{if } w \cdot x + b < 0 \end{cases} \tag{3.3}$$

Logic Functions

With the correct parameters selected for weights and threshold, the concept of the perceptron can be used to implement logic functions like OR, AND or NAND (Figure 3.3). A NAND function has universal completeness, meaning all boolean function can be converted to a combination of NAND gates. With enough NAND gates available, every combinatorial logic function can be realized [33, p. 62].



x0	**x1**	O
0	0	1
1	1	1
1	0	1
1	1	0

Figure 3.3: Parameterization of a perceptron to implement the NAND logic function. Following [34].

This fact makes it possible to build any combinatorial logic function out of perceptrons. Since those combinatorial logic functions are the fundamentals of computer science, all concepts of computer science can be represented through the usage of perceptrons, which makes the concept of perceptrons universal for computation. A network of perceptrons can represent any other computing device like an adder, calculator or a modern CPU.

Figure 3.4 is showing an implementation of a half adder using only NAND logic gates. By building a network out of perceptrons parameterized to fulfill the NAND logic function, a half adder can also be composed from perceptrons (see Figure 3.5)

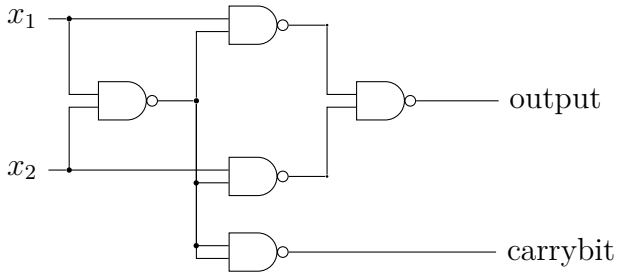


Figure 3.4: Implementation of a full adder using only NAND logic gates. Following [34]

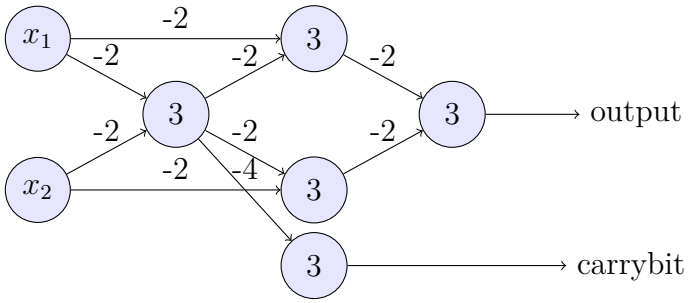


Figure 3.5: Implementation of a full adder with perceptrons. For simplification, the bias is written into the perceptron and the weights are above the connections. Following [34]

The network of perceptrons comes with the advantage of being able to adapt to any other logic function by simply changing its weights and biases. Thus, a network of perceptrons can be parameterized to response to any external stimulus given.

Learning Perceptrons

With the perceptron being adaptable to any logic function, the use of linear algebra enables us to determine the parameters needed to fulfill any given logic function [33, pp. 78-84]

By converting the function describing the perceptron to have a fixed input and be variable on the weights and bias, an input vector and desired output can be applied to the equation and weights and bias can be calculated. The perceptron can learn a new function by adjusting its weights and bias in situations where a complex network of conventional logic functions might be suited.

The biggest drawback in learning with a perceptron is the binary output of the step function. Small changes in the weights and bias will not result in any changes of the

output. Only reaching the threshold will result in changes of the output. So small changes will either result in no change at or they will result in a full change of the perceptron's behavior. In a network of perceptrons small changes on weights might cause the network to work in an unpredictable way.

3.1.2 Artificial Neurons

Modern neural networks use a modified version of a perceptron which shares the same fundamental concepts, but because of small modifications to the algorithm it can be adapted, so that small changes in weights and bias will result in changes to the output value.

The perceptron's threshold is converted into a bias with $b_j \equiv -\theta$. The bias can be implemented as one additional input $x_{n+1} = b$ and an attached weight with value $w_{n+1} = 1$ So all calculations in form $w \cdot x + b$ can be simplified to $w \cdot x$. The neuron is also modified to accept non-binary input data.

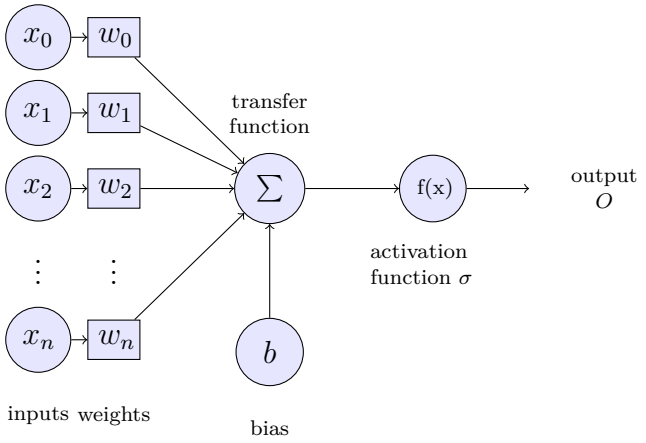


Figure 3.6: A schematic representation of an artficial neuron

This changes the threshold from being an input variable of the step function to a normal weighted input of the neuron. Like this the activation function can be replaced by any other function in the form $\sigma = f(x)$ With any function working as activation function, small changes in weights Δw can result in changes in the output ΔO.

An activation function that is commonly used within artificial neurons is the sigmoid function which is define by equation 3.4. With the sigmoid function used as activation

function, the equation for the output of an artificial neuron is defined by equation 3.5

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{3.4}$$

$$O = \frac{1}{1 + \exp\left(-(\sum_j (x_j * w_j) + b)\right)} \tag{3.5}$$

The sigmoid function is behaving similar to the step function, so the behavior of the neuron is not changing to one of a perceptron. The higher the exponent value in the sigmoid function gets, the more it is approximating a step function, as shown in figure 3.7 The higher $x \cdot w + b$ gets, the more a neuron is converging to a perceptron.

Small changes in Δw_j shall result in small changes in ΔO. Using calculus ΔO can be determined by equation 3.6. This shows that ΔO is a linear function of Δw and Δb. The learning process of an artificial neuron is thus aiming towards determining Δw by measuring ΔO.

$$\Delta O = \sum_j (\frac{\partial O}{\partial w_j} \Delta w_j) + \frac{\partial O}{\partial b} \Delta b \tag{3.6}$$

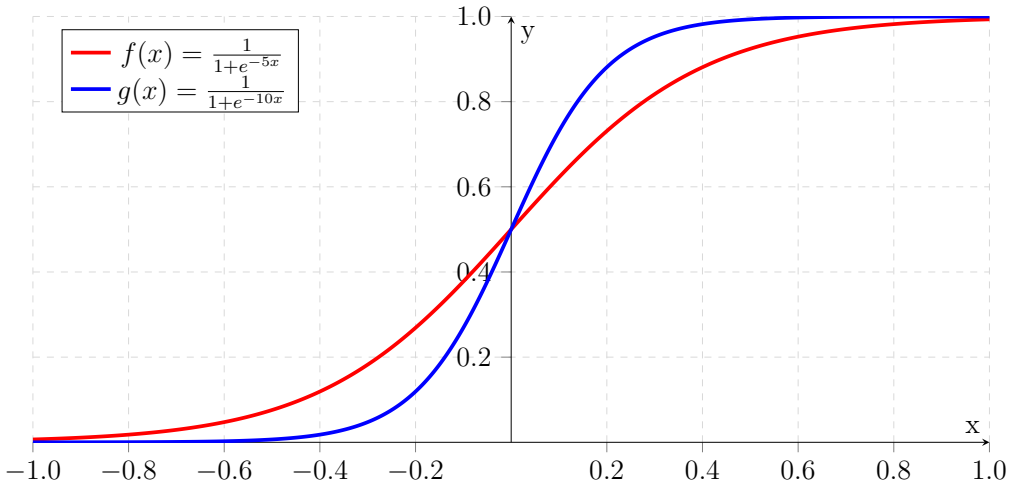


Figure 3.7: Plot of sigmoid function

3.1.3 Networks of Neurons

Analogous to the concept of connecting perceptrons illustrated in figure 3.5, neurons can be connected to build a network. The neurons can be connected in different combinations. The following paragraphs will introduce the most common network architectures in recent research.

Feed Forward Network

A common combination is to organize the neurons in layers, where all outputs of one layer are connected to all inputs of the next layer, as depicted in figure 3.8 . This type of network is called a *feed forward neural network*. The first layer is containing the input values of the problem that will be solved, the last layer represents the output. They are called *input layer* and *output layer*. The layers between input- and output layer are called *hidden layers* [33, pp. 125-148].

The organization of input- and output layer are directly mapping to the problems input and output variables, the design of the hidden layers can be more complex. A layer A consisting of n neurons connected to a layer B consisting of m neurons is resulting in a total of $n * m$ connections between those layers. Each neuron in layer B is connected to n neurons from layer A, resulting in n inputs and thus n weights connecting to this neuron.

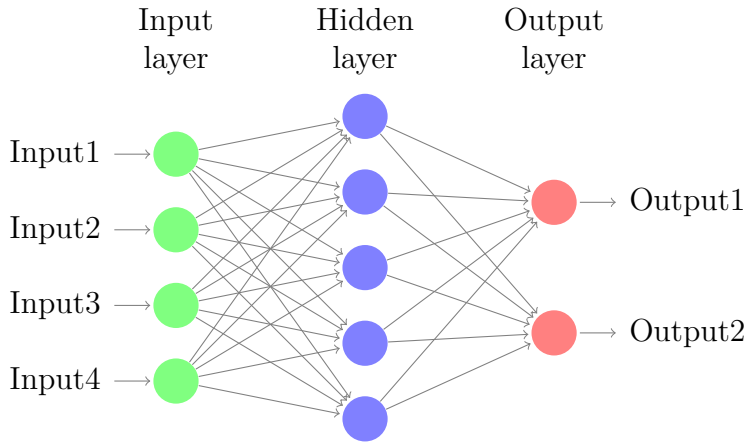


Figure 3.8: Graph of a feed forward neural network with a single hidden layer.

Recurrent Networks

While in feed forward networks the connections from one neuron only connect to neurons of the next layer, recurrent networks also implement feedback loops, where the output of one neuron can be fed back as input of another neuron of a previous layer, or even connect to its own input, as illustrated in figure 3.9.

Their structure of applying feedback loops makes them useful on long short-term memory based models, like acoustic analysis [35] However, in practice recurrent networks researchers face problems in applying learning algorithm like the back-propagation algorithm to them, because the feedback loops can easily lead to exploding or vanishing gradients [36].

Their behavior is very complex and can become unpredictable. This thesis will not concentrate on those kind of networks.

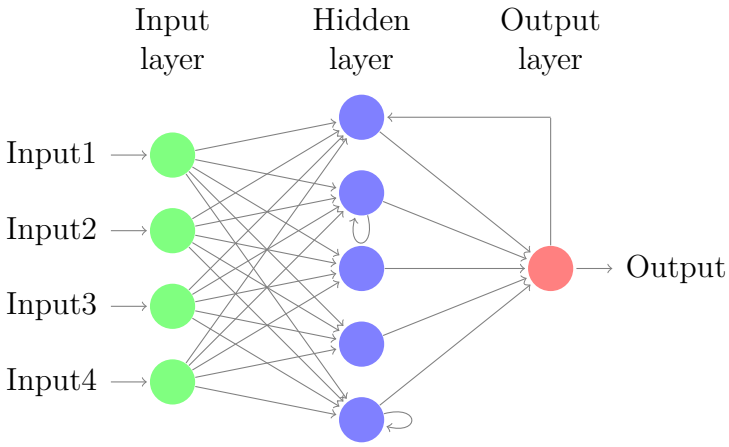


Figure 3.9: Graph of a recurrent neural network.

Hidden Layer Organization

The number of hidden layers and the amount of neurons in those layers is dependent on the available data, meaning the number of input and output neurons, the amount of training data available, the complexity of the underlying problem and the algorithm used for training the network [33, pp. 125-148].

Number of Hidden Layers

Linear separable problems can be solved without the need for any hidden layers, but more complex problems require at least one hidden layer to be solved. As the

universal approximation theorem states, a feed forward network using only a single hidden layer with enough neurons can approximate to any function given [37]. While this allows to theoretically solve any given problem with just one hidden layer, the theorem does not state the possibility of training the network. If it empirically proves to enhance the performance for the given problem, the usage of 2 or more hidden layers can be useful. A network with multiple hidden layers is illustrated in figure 3.10.

When multiple hidden layers are present in a neural network, a layer can be understood as pre-filtering the data fed to the next layer, where groups of neurons of one layer are filtering specific characteristics of the dataset. The next layers will make decisions based on these characteristics, not the underlying data.

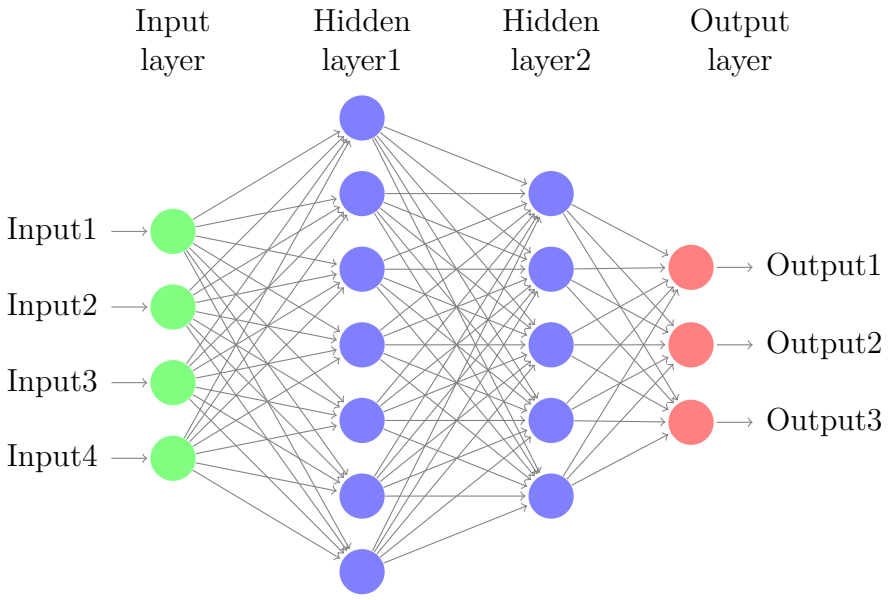


Figure 3.10: Graph of a feed forward network with multiple hidden layers.

Number of Neurons in the Hidden Layers

Involving too few neurons in the hidden layers will cause the network to not correctly approximate the problems solution, leading to a underfitting network. In contrast involving too many neurons will cause the network to overfit by solving problems that were not required to solve and thus can cause the network to not adapt to data not present in the training sets. The relation between under- and overfitting networks is depicted in figure 3.11.

While some rules of thumb exist that set the number of neurons around the range of $\sqrt{n_{in} * n_{out}}$, where n_{in} and n_{out} describe the number of input- and output units

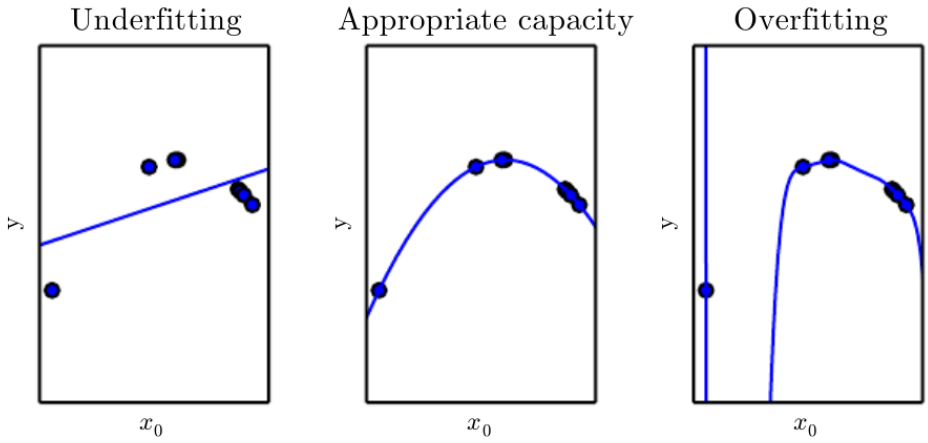


Figure 3.11: 3 models of a neural network solving the problem of mapping points from a quadratic function. **Left**: A linear model is leading to underfitting by not accurately capturing the given data samples. **Right**: A polynomial model can suffer from overfitting by passing all points, but not leading to the optimal structure. **Center**: A quadratic model is fitting the data well, even to unseen points. Image and description reused from [38, Ch. 5, p. 111].

connected to the layer [39], the exact number cannot be estimated mathematically and must be solved empirically by observing the networks error during training and validation [40].

Convolutional Neuronal Networks

Convolutional neural networks were introduced in the 20th century [41, 42] and were widely adapted for classification tasks [43] They got public attention after the proposal of AlexNet, a convolutional network used for image classification [14]. Convolutional neural networks are a specialized version of conventional neural networks, based on the assumption that the input data is made of images.

While ordinary neural networks, like feed forward networks work well on small images, they do not scale well for bigger image size with multiple colors, as their number of weights from input layer to the first hidden layer will grow very large, which would increase the overall network size as well, which eventually will lead to become unpracticable.

Convolutional neural networks arrange their neurons in a 3-dimensional structure, based on the images width, height and depth, where the layers are not fully connected: A layer only connects to a small part of its previous layers neurons as illustrated in

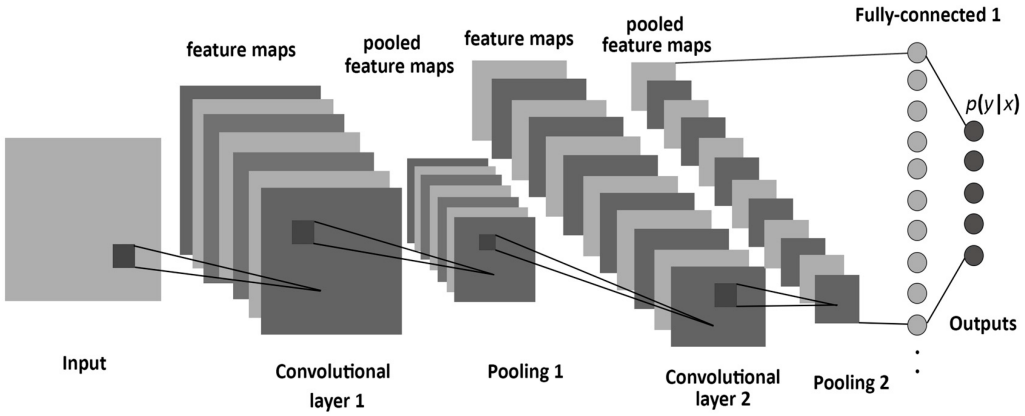


Figure 3.12: Architecture of the LeNet-5 [43], a convolutional neural network. Image reused from [44].

figure 3.12. This area is slided over the full surface of the image, a mathematical operation called convolution.

The layers in a convolutional network can vary in their function and parameters to control them: While convolutional layers slide over all neurons of the previous layer, pooling layers combine the output of multiple neurons into a single output neuron.

3.1.4 Learning as a Gradient Descent

As shown in previous sections, neural networks are capable of approximating a wide range of functions by implementing hidden layers with an adequate amount of neurons parametrized by their weights and biases. This section will discuss a method that enables the network to learn those parameters by feeding inputs and the desired outputs into the network.

To quantify the correctness of the networks approximation, the cost function defined by equation 3.7 is used. This equation is called the *mean squared error* or *quadratic cost function* C. It is a function dependent on the weights and biases of the network. The difference between the desired output a and the networks output y is built for all inputs x. Their number is determined by n.

$$C(w,b) = \frac{1}{2n} \sum_{x} |y(x) - a|^2 \tag{3.7}$$

The goal of a learning the parameter to approximate any function correctly is to change the weights and biases in a way that the cost functions output is as small as possible. The closer $C(w,b)$ gets, the better the function will be approximated.

This can be achieved by a technique called *stochastic gradient descent* or SGD, an iterative method to solve minimization problems [45].

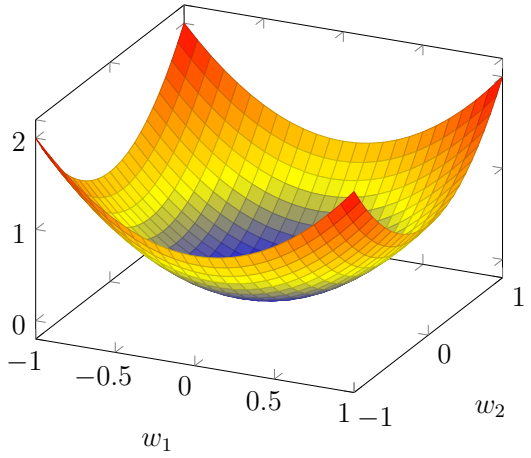


Figure 3.13: The surface plane of the Mean Squared Error with 2 inputs

With stochastic gradient descent small changes Δw and Δb are incrementally applied to the cost functions parameters w and b, until its output is minimized to satisfaction. Figure 3.13 illustrates the surface pane of the cost function for 2 inputs. The SGD method can be imagined as rolling a ball on the pane until it reaches the bottom. The lowest point is called the global minimum of the cost function and is considered the optimum solution to the minimization problem.

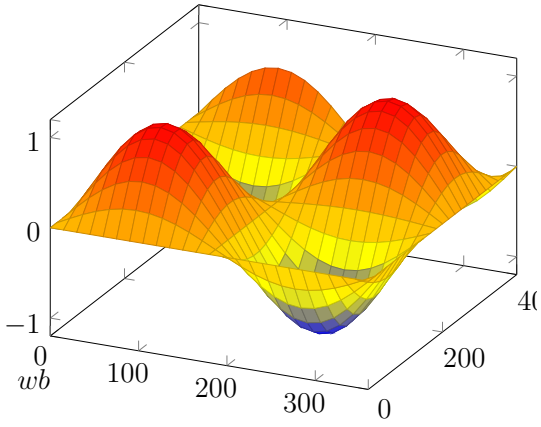


Figure 3.14: A surface plan of a network showing local and global minima.

While 3.13 depicts a function dependent on just 2 input parameters and the global minimum is easily comprehensible, the cost for complex problems is dependent on thousands of parameters, resulting in a more complex pane as illustrated in 3.14. This pane can have multiple local minima, which could lead to incorrect assumptions of

the best solution [33, pp. 101-119]. Avoiding local minima is a common problem during the training of neural networks.

A common technique to overcome this problem is to change the neurons activation function. While the sigmoid function will map negative inputs to outputs close to zero, the hyperbolic tangent, as illustrated in figure 3.15, will map negative inputs to negative outputs, which leads to a wider range of output values. As figure 3.16 illustrates the hyperbolic tangent produces stronger gradient, which will lead to bigger Δw that are leading the SGD less likely to remain in a local minimum [46].

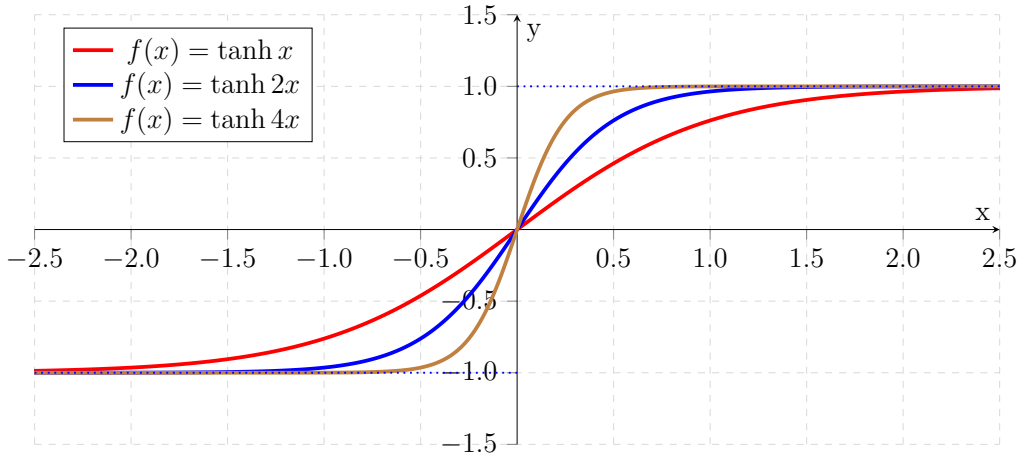


Figure 3.15: Plot of the hyperbolic tangent.

The cost C can be minimized by the iterative process of the gradient descent for which the gradient of w is calculated as shown in equation 3.8. Each weight is updated by Δw, defined in equation 3.9, where η is a constant for the learning rate, a parameter that scales the learning process [33, pp. 101-119].

$$\Delta C = \frac{\partial C}{\partial w_1} + \frac{\partial C}{\partial w_2} + ... \frac{\partial C}{\partial w_n} \tag{3.8}$$

$$\Delta w = -\eta \frac{\partial C}{\partial w_i} \text{ for } i = 1, 2, 3, ..., n \tag{3.9}$$

As show in equation 3.9 Δw is depending on ΔC, while C is depending on the activation functions output a. To determine Δw the activation function must be differentiable. Equation 3.10 is defining derivative of the sigmoid activation, 3.11 the derivative of the hyperbolic tangent.

$$s'(x) = \frac{\partial}{\partial x} s(x) = s(x)(1 - s(x)) \tag{3.10}$$

$$tanh'(x) = \frac{\partial}{\partial x} tanh(x) = 1 - tanh(x)^2 \tag{3.11}$$

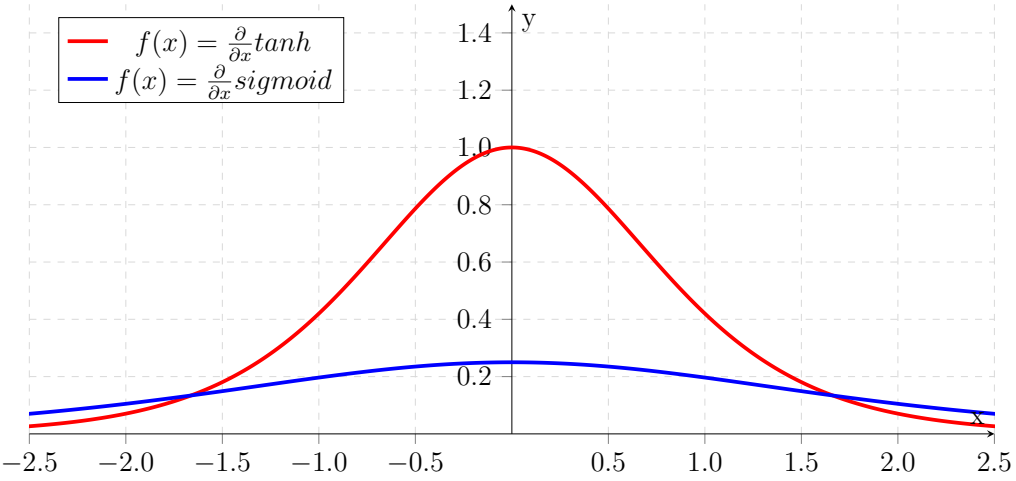


Figure 3.16: Partial derivative of sigmoid and hyperbolic tangent.

3.1.5 Backpropagation Algorithm

The method of stochastic gradient descent, described in 3.1.4, is applying small changes to the networks weights in order to approximate a desired output. The exact amount of change to each weight is defined by the partial derivate of the networks cost function with respect to the weight examined. It is called the *gradients* of the weights vector.

The backpropagation algorithm, developed in 1988, is a method to determine these partial derivatives by calculating the error for each output neuron and propagating it backwards through the network [47].

The network can be understood as a chain of function compositions, where each node of the network is given a composite structure: The right side of the node computes the neurons activation function, while the left side computes the activation function derivative for the same input, as depicted in figure 3.17. The result computed at the right side is considered the output of the neuron and sent to the neurons connected to it, while the result computed at the left side is stored in the neuron itself [33, pp. 152-182]. During a second step, the backpropagation, where the network is propagated backwards, the previously stored results are used as the neurons outputs.

A simplification to a neuron during backpropagation can be made: A neuron is a composition of an integrator +, summarizing the weighted inputs and an activation

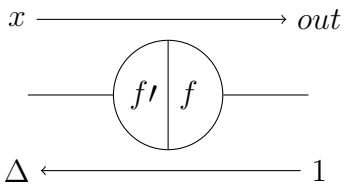


Figure 3.17: Function derivates during network propagation. Image following [33, p. 158].

function s, as shown in figure 3.18. When building the derivate of both nodes, the activation functions derivate get s', while the integrators derivate is 1 for all inputs, allowing us, during backpropagation, to reduce the neurons calculation to only its activation function.

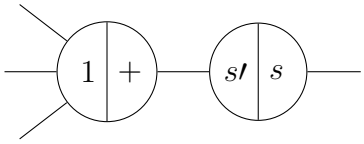


Figure 3.18: Function derivatives during network propagation. Image following [33, p. 158].

To apply backwards propagation through the composited network, three cases have to be considered: Function composition, addition and inputs weighting must be proven to be generalizable in order to be able to formulate the algorithm for backpropagation. They will be discussed in the following paragraph.

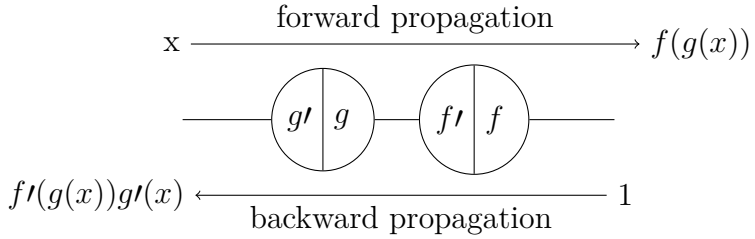


Figure 3.19: Function composition during forward and backward propagation. Image following [33, pp. 159-160]

The **composition** of the functions f and g is visualized in figure 3.19. During forward propagation the input x is fed to the left side of the network. The nodes output and its derivative are computed. The derivative is stored in the node, while the output is fed to the next node. The networks output is the composition $f(g(x)$.

Running the network backwards by feeding 1 to the right side of the network, in each node the input is multiplied by the value stored in the unit. Thus, the derivative of the function composition $f(g(x))$ is $f'(g(x)g'(x)$. Since the backpropagation implements the chain rule this result can be generalized to any sequence of function compositions [33, pp. 159-160].

Figure 3.20 depicts the **addition** of the functions f and g. As already shown for the simplification of the neurons integrator, the node implementing the addition has a partial derivate of 1 for all inputs. When forward propagating the network, the network computes $f(x) + g(x)$ and the derivatives $f'(x)$ and $g'(x)$ are stored inside the nodes. During the backpropagation step a 1 is fed to the right side of the network, multiplied with the partial derivatives stored in the nodes, the result of the step is $f'(x) + g'(x)$. As shown in [33, pp. 159-160], the result can be generalized to any addition of functions.

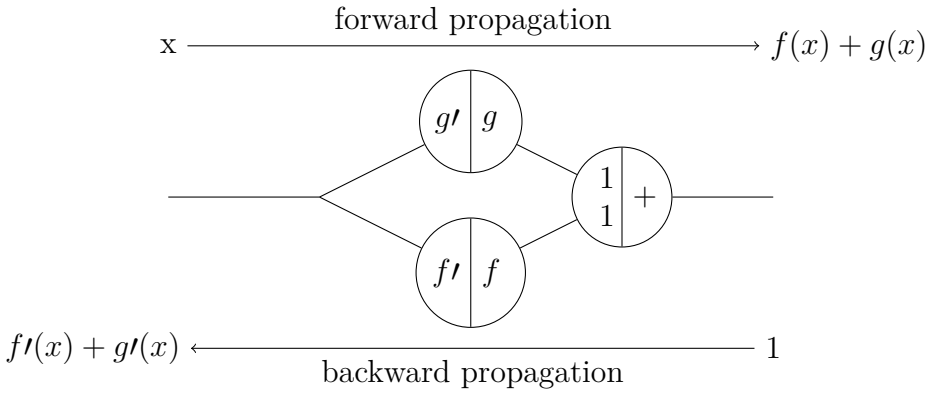


Figure 3.20: Function addition during forward and backward propagation. Image following [33, pp. 159-160]

Weighted inputs can be understood as multiplying each sides input with w: While during forward propagation the nodes input x is multiplied with the weight w, resulting in wx, during backward propagation the weight is multiplied by 1 fed from the right side of the node, resulting in w. This is the derivative of wx with respect to x. Thus, it can be concluded that weights are modulating the nodes information from both sides, by multiplying each sides input with w [33, p. 161].

Steps of the Backpropagation Algorithm

The backpropagation method is defined by algorithm 1.

Algorithm 1 Backward propagation.
N is the amount of training data, L the amount of layers and t the training vector. a is the vector of activations, w the vector of weights. C is the cost and η the learning rate.

for $n = 1$ to N **do**
  1. propagate forward
  for $k = 1$ to L **do**
    $a_k \leftarrow dotproduct(a_k, w_k)$
  end for
  2. backpropagate to output layer
  $\Delta w_L \leftarrow -a'_L * (a - t)$
  3. backpropagate to hidden layers
  for $k = L - 1$ to 1 **do**
    $\Delta w_k \leftarrow -\frac{\partial C}{\partial w_k}$
  end for
  4. update weights
  $w \leftarrow w - \eta \Delta w$
end for

1. Feed forward

In a first step, the network is propagated forward by applying the dot product of weights and activations for each layer as described in 3.2. Additionally, the evaluated derivatives of each neurons activation function for are stored in the unit [33, p. 161].

2. Backpropagate output layer

During backpropagation to the output layer, the partial derivatives $\frac{\partial C}{\partial w}$ are calculated and for each neuron multiplied by its evaluated error as shown in in equation 3.12 [33, pp. 167-168].

$$\Delta w_{i,j} = \frac{\partial C}{\partial w_{i,j}} * c_i \tag{3.12}$$

3. Backpropagate Hidden Layer

While the error of output neurons can easily be calculated, for the hidden layers, this error for each neuron is dependent on all neurons connected to it. The neurons error

is calculated by equation 3.13 [33, p. 169].

$$\Delta w_{i,j} = \frac{\partial C}{\partial w_{i,j}} * \sum_{q} w_{i+1,q} * C_q \tag{3.13}$$

4. Update Weights

In the last step all neurons weights are updated by applying the previously calculated Δw to them. A constant, the learning rate, is used to scale the step size of the correction to the weights [33, p. 170].

If there is more training data available, the process can be repeated and weights corrections can be applied to each set of training data, resulting in the network being able to approximate all patterns available in the training data.

3.1.6 Binarized Neural Networks

Binarized neural networks (BNN) are a recently introduced variant of neural networks with weights and activations represented as binary values with the intention to drastically reduce memory size and accesses and as well reduce the computational cost by replacing arithmetic operations by bit-wise operations [1].

3.1.6.1 Binarizing Weights and Activations

In binarized neural networks weights and activations are constrained to the values -1 or 1. Binarizing the networks weights can be performed by transforming the real-valued variables into the binary values [48].

This can be performed in either a deterministic or a stochastic way. While the stochastic binarization shown in paragraph is resulting in a higher accuracy during classification, it requires the generation of random values to calculate the probability. In a computational inexpensive environment, the generation of random values is not desired, since it is lowering the networks performance. The deterministic avoids the generation of randomness and is thus the preferred implementation.

Deterministic Binarization

With deterministic binarization the networks weights are transformed by the *sign* function shown in equation 3.14. Figure 3.21 is showing the graphical representation of the sign function.

$$f(x) = \begin{cases} 1 & if\ x \geq 0 \\ -1 & if\ x < 0 \end{cases} \tag{3.14}$$

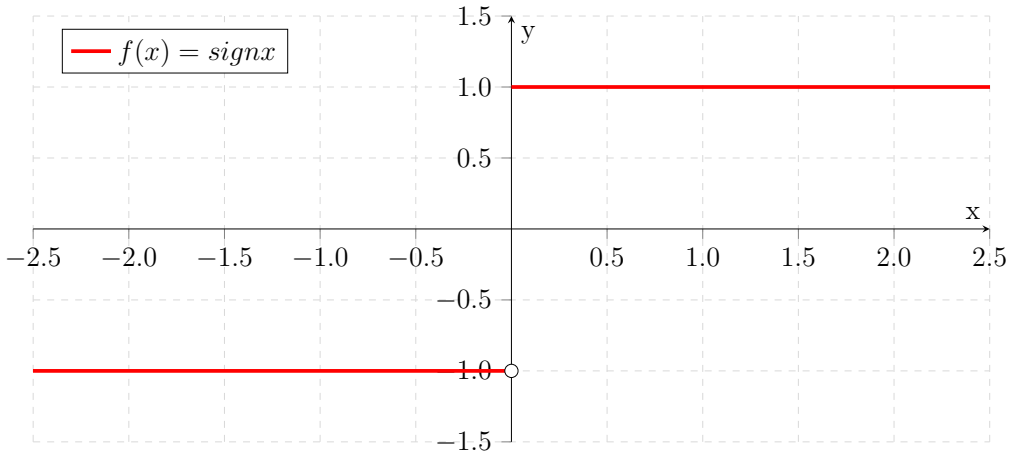


Figure 3.21: Plot of sign function

Stochastic Binarization

In stochastic binarization the variables are transferred with equation 3.15, where $\sigma(x)$ is the *hard sigmoid* function, defined in equation 3.16.

$$f(x) = \begin{cases} 1 & \text{with propability } p = \sigma(x) \\ -1 & \text{with propability } 1 - p \end{cases} \tag{3.15}$$

$$\sigma(x) = clip(\frac{x+1}{2}, 0, 1) = max(0, min(1, \frac{x+1}{2})) \tag{3.16}$$

3.1.6.2 Binarized Forward Propagation

In a binarized neural network all weights and activations are constrained to -1 or 1. This leads to possible optimizations in the calculation of a neurons activation. While in conventional networks the activation vector is multiplied with the weight matrix by applying the dot product, the operation can be simplified to a bitwise XNOR operation [2] in binarized neural networks as shown in table 3.1.

x0	**x1**	$\bigotimes$
0	0	1
0	1	0
1	0	0
1	1	1

(a) XNOR LUT with inputs 0 and 1.

x0	**x1**	$\bigotimes$
-1	-1	1
-1	1	-1
1	-1	-1
1	1	1

(b) XNOR LUT with inputs -1 and 1.

Table 3.1: Lookup-Table for the XNOR function.

Equation 3.17 defines the forward propagation procedure on binarized neural networks [3], where l is the index of the layer, i is the index of the neuron calculated, a is the neurons activation and b the neurons bias. K is the amount of neurons in the layer, w the neurons weights. The $\bigotimes$ symbol describes the bitwise XNOR operation.

$$a_i^l = b_i^l + \sum_j^{K^{l-1}} w_{i,j}^l \bigotimes a_j^{l-1} \tag{3.17}$$

The forward propagation step is similar to those of conventional networks with the exception that the dot product operation on activations and weights is replaced by the XNOR operation. Since the first layer is containing real valued inputs derived from the dataset under investigation, it must be preprocessed by binarizing the values. Depending on the range the input parameters are, the exact way of binarization is not defined and must be adapted to the given range of input parameters.

In the case of the MNIST dataset, the input parameters range from 0 to 255, representing grey-scale images. The inputs can be binarized by applying a threshold of 80, following equation 3.18.

$$a_i^b = \begin{cases} 1 & \text{if } a_i \geq 80 \\ -1 & \text{else} \end{cases} \tag{3.18}$$

3.1.6.3 Binarized Backward Propagation

During backpropagation the activation functions derivative is used. For the *sign* functions the derivate is zero on all points, except the zero point of the function, where the derivative is undefined, which makes it incompatible with the backpropagation algorithm.

Algorithm 2 defines the process for backward propagation of a binarized neural network. It is structured analogous to the backward propagation of conventional networks, but is applying modifications in order to overcome the disadvantages binarized weights and activations have to the algorithm.

Algorithm 2 Backward propagation in a binarized neural network.
Model derived from [1].
N is the amount of traing data, C is the cost function for the dataset, η is the learning rate, L the number of layers. w are weight vectors, b are biases vectors, a the activation output.

for $n = 1$ to N **do**
  1. binarize input
  for $i = 1$ to n **do**
    $a_i^b \leftarrow thresh(a_i)$
  end for
  2. forward propagation
  for $k = 1$ to L **do**
    $w_k^b \leftarrow binarize(w_k)$
    $a_k \leftarrow xnor(a_k, w_k^b)$
    $a_k^b \leftarrow sign(a_k)$
  end for
  3. compute gradients
  for $k = L$ to 1 **do**
    Compute $\frac{\partial C}{\partial w_k^b}$
    Compute $\frac{\partial C}{\partial b_k^b}$
  end for
  4. update parameters
  for $k = 1$ to L **do**
    $\Delta w_k \leftarrow \eta \frac{\partial C}{\partial w_k^b}$
    $w \leftarrow clip(w_k - \Delta w_k^b, -1, 1)$
    $b \leftarrow b_k - \eta \frac{\partial C}{\partial b_k^b}$
  end for
end for

The networks input layer is binarized and forwards propagation is applied to the network as defined in 3.1.6.2. The gradients are calculated with respect to the partial derivatives of Cost and weights as described in 3.1.5. In this step both, the binarized and real-valued gradients are kept for further processing. When updating the parameters the gradients are applied to the real-valued weights and biases. To avoid exploding gradients, the updated parameters are clipped at -1 and 1 [48].

3.2 Parallel Computing Platforms

Neural networks can be implemented on different platforms that offer sequential or parallel computing structures. This section gives an overview of the platforms commonly used for processing neural networks and other computer vision tasks.

3.2.1 Overview of Platforms

3.2.2 Challanges for Parallel Hardware Architectures

GPU and FPGA based accelerators are working well for processing parallelizable tasks like the computation of neural networks. While both perform well in accelerating parallel processes, they implement parallelism very differently.

The following examples are showing main differences in common use cases met with parallel processing applications on embedded hardware.

Figure 3.22 shows the difference between SIMD and pipelined implementations in hardware. Five instructions (A-E) are executed on six work items (1-6). The SIMD can handle 3 work items at the same time, while the pipeline structure adapts to the amount of instructions and work-items needed. Processing the example, the GPU implementing SIMD strategy completes 3 work items every 5 clock cycles. The pipelined FPGA structure can, after initial 5 clock cycles, finish one work item each clock cycle by executing multiple instructions in parallel.

Figure 3.23 illustrates the difference in handling branching. While within the FPGA all possible code paths are already established in hardware before execution, the execution is not different from the previous example and one work item each clock cycle can be achieved. Within the GPU on the other hand only handles a single instruction at the same time, which leads to conditional execution of the single branches and thus can lead to conditional enabling or disabling of specific work items during execution. This behavior can in the worst-case result in an execution time that is equal to one without parallel strategies implemented.

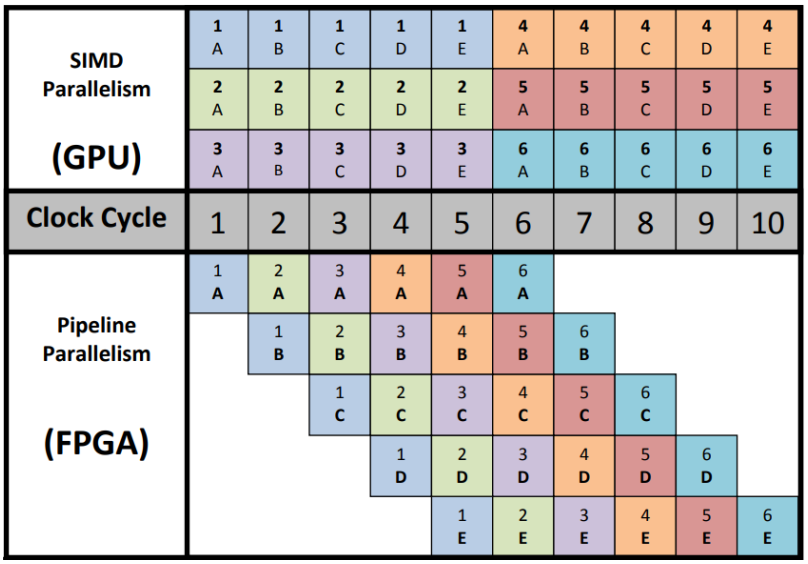


Figure 3.22: SIMD versus pipelined architecture. Image reused from [49].

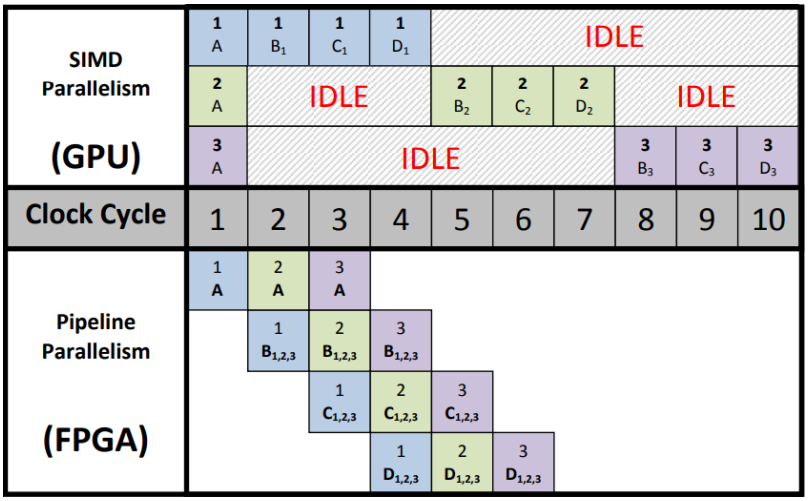


Figure 3.23: SIMD versus pipelined branching. Image reused from [49].

Figure 3.24 depicts an advantage SoC integrated FPGA can offer compared to GPUs. While with a GPU usually all accesses to IO peripherals go through the host system, a FPGA embedded in a SoC device has direct access to the connected peripherals and thus, the FPGA fabric can access IO peripherals without dealing with the host system.

Even in executing non parallel tasks, like single work-item kernels, FPGAs can profit from their pipelined structure. As Figure 3.25 shows how single instructed can be pipelined while executing a loop. All instructions can be executed in parallel, where

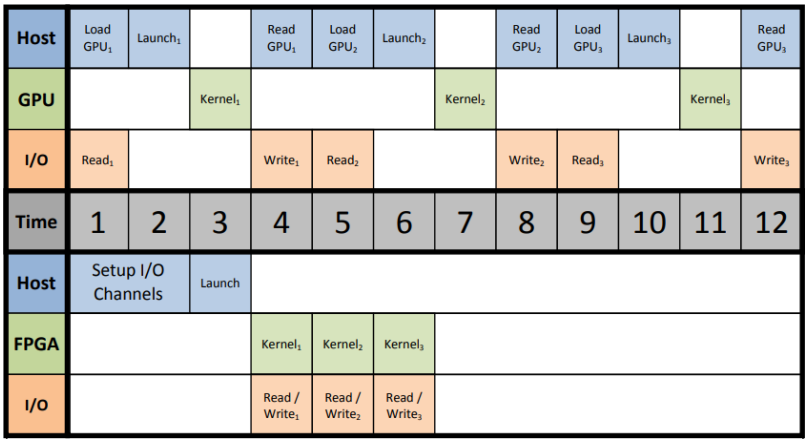


Figure 3.24: SIMD versus pipelined IO access. Image reused from [49].

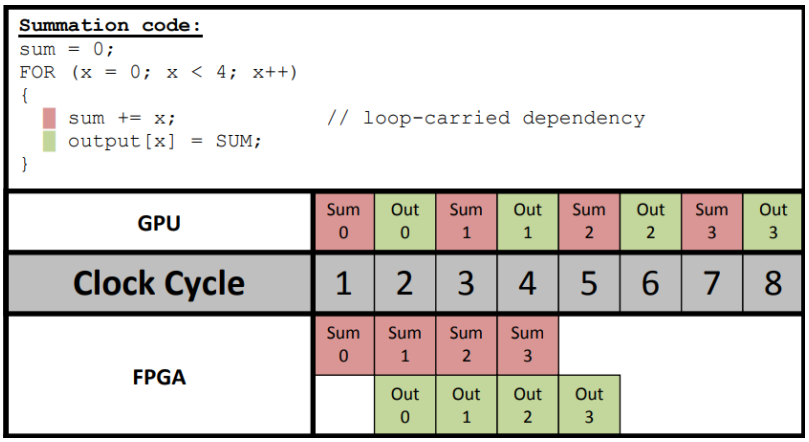


Figure 3.25: SIMD versus pipelined loop execution. Image reused from [49].

the data executed in loop iterations work as separate work items.

However with the usage of OpenCL FPGA hardware is also partly used in a SIMD structure: When a kernel is implemented as a NDRange kernel, the execution of the instances of this is also performed in a SIMD structure, as it is done with GPUs.

3.3 Performance Indicators

TODO: rephrase a little

Comparing different neural networks and their implementation can become a difficult task. They may vary in the number of neurons and hidden layers and measurements

can be taken on different architectures. To be able to compare different implementations processing the same task, it is needed to define measurable indicators that can distinguish different features of the network.

3.3.1 Classification Accuracy

The most important factor in the process of classifying images is the percentage of images classified correctly. The classification accuracy is highly dependent in the training data provided, so it is important to only compare networks that were trained with the same datasets.

A good accuracy is considered 80%. An accuracy of 84.4% was achieved in the ImageNet competition by the AlexNet network [14], which is the basis of all modern convolutional network implementations.

3.3.2 Execution-Time per Frame

Besides how accurate a network is, another important factor is the time the network needs to process one image. This gives a rough estimation on the implementations performance. Since different authors may implement their work on different platforms, it should only work as a measurement for implementations on the same platform and the same training dataset.

3.3.3 FPGA Logic Gates used

One big factor for any algorithm that is supposed to run in an FPGA device is: Will it fit into the FPGA? FPGAs have a limited amount of different resources, like lookup tables, logic gates or DSP blocks. After synthesizing the FPGA logic, the clock might not be sufficient anymore, or the logic needs more resources as the chip offers. Since FPGA prices are growing exponentially with the amount of resources they offer, it becomes important to be able to fit the logic into chip that fit into the projects budget. In mass production it is useful to keep the hardware costs as minimal as possible, because those costs will reoccur and will also increase the more devices are sold.

3.3.4 Development Cost

In most commercial project their owner wants to push the costs to minimum. Cost for a software project can be measures in lines of code (LOC) that are needed to finish the project. An amount of LOC can be estimated that a developer can write during one day. This amount is different for different programming languages: Programmers using C or C++ usually write less code a day as programmers writing in Python do [50, pp. 55-56].

3.3.5 Training Cost

Since training is usually done prior to the shipment to the end user, the time needed to train the network does not affect the experience the user will have with the network. However long training times increase the costs and are getting a big factor when it comes to learning new datasets after deployment of the network or even during runtime.

Thus, it is useful to have low training cost. The cost of training can be estimated by the time the model needs for training.

Since the time can vary enormously between the implementation on different processor architectures, measurements should only be compared, if they were performed on the same hardware.

3.3.6 energy consumption battery life

Energy consumption of the system is very important to the end user. Usually implementations on FPGAs are consuming less energy as the same implementation would consume on a GPU. Since the author does not have access to all implementations discussed in this thesis, only comparisons between the used architectures can be chosen for the comparison.

4 System Architecture

TODO: rephrase a little - maybe NOT explain HPS structe, but rather how fpga can connect to all the peripherals etc

This chapter discusses the hardware and software architecture used in the implementation of this thesis' work.

4.1 Network Structure

The neural network implemented during the research of this thesis is based on a feed forward network structure.

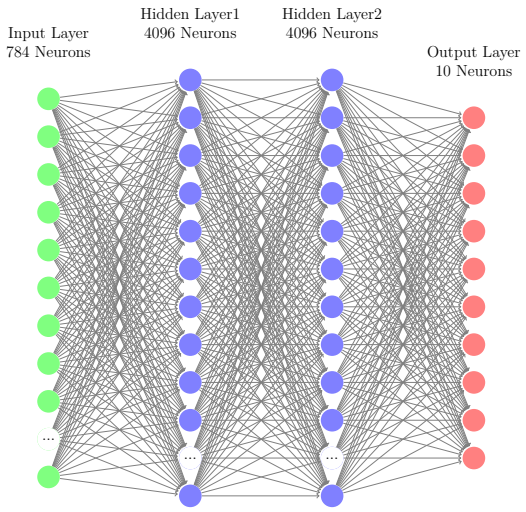


Figure 4.1: Graph of the implemented network. The network consists of 384 input neurons, 2 hidden layers with each 4096 neurons and an output layer of 10 neurons.

While usually convolutional networks are used for image classification tasks and show best performances, they tend to show best performances when implemented as very deep networks, like AlexNet [14] or even ResNet [17]. The implementation of such big networks in low-level programming languages as well as the huge training periods

they require, would consume more time as what is allocated for the practical work of this thesis. All expected results that can be derived from investigating binarized feed forward networks can easily be transferred to other network types. Considering the limited hardware on which the practical work of this thesis is supposed to run the decision was made towards the implementation of a feed forward network.

Network Layout

Figure 4.1 illustrated the network architecture. The **input layer** consists out of 384 neurons. The EMNIST dataset is made of images of $27 * 27$ pixels of grey scale values ranging from 0 to 255. Each pixel can be translated to one input neuron in a network, which results in 784 input neurons. The pixels grey scale value is scaled into a range from -1 to 1 and is afterwards binarized to the values -1 and 1.

The **output layer** is made from 10 neurons with binarized output values of -1 or 1. The goal of the network is to be able to classify number from 0 to 9, resulting in 10 classes, each represented by a single neuron in the output layer.

Input- and output layer are connected through one or more **hidden layers**. While with a conventional network one hidden layer with less than 1000 hidden neurons is needed show acceptable accuracy when classifying digits, binarized networks require a minimum of 2 layers with 2048 neurons to reach an accuracy of more than 50%. Results with an accuracy over 85% could only be reached by using 2 layers of 4096 neurons.

4.2 OpenCL Integration

Intel's SoC platforms can be used with OpenCL through the OpenCL SDK provided by Intel.

Figure 4.2 illustrates the program flow of classifying one image. The ARM core starts by reading the image from DDR memory, and filling the input layer of the neural network. For each layer it writes the data needed for processing that layer to DDR and initiates the FPGA acceleration of the calculation of the dot product.

For simplification, this illustration depicts the processes executed inside the FPGA as one single process, while in fact multiple processes are executed in parallel inside the FPGA. A more detailed view of those processes is illustrated in 4.3.

The FPGA is reading the required data and writing back the answer to DDR memory. Afterwards it informs the ARM core about finishing the processing. The ARM core

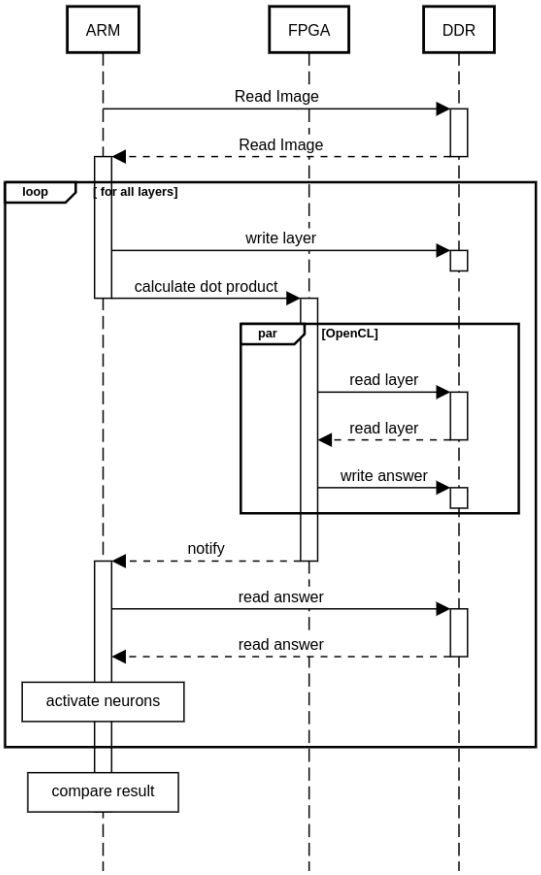


Figure 4.2: Sequence Diagram for ARM and FPGA cores accessing DDR.

reads back the answer and is finishing applying the activation function to the dot product calculated by the FPGA core. After all layers of the network are processed, the result in the output layer is verified to be the correct representation of the data.

While in figure 4.2 the processing done in the FPGA is represented as a single thread, figure 4.3 is providing a deeper view into the scheduling inside the FPGA. The FPGA is calculating the dot product of a matrix of weights and a vector of activations. To achieve this, each row of the matrix is multiplied with the activation vector and the results are added.

Internally, for each row of the matrix an OpenCL kernel is executed, execution the multiplication and addition and writing the result back to the corresponding cell in the output vector. Those threads are executed in parallel and are scheduled by the OpenCL scheduler subsystem. Each thread is accessing only the row in the matrix that it needs access to, by executing its own DDR memory accesses.

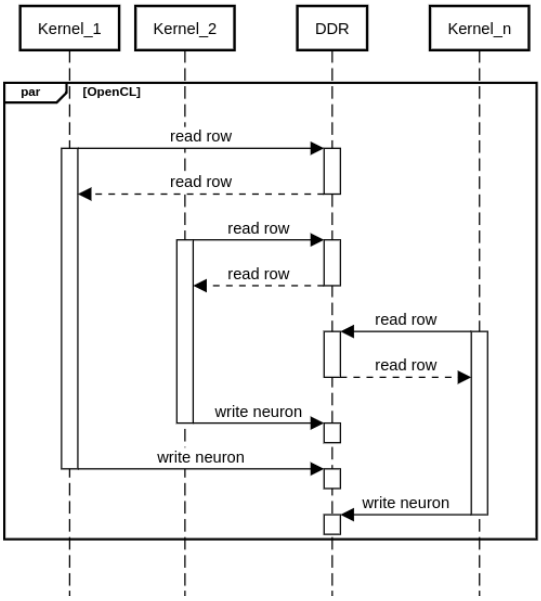


Figure 4.3: Sequence Diagram for OpenCL Kernels accessing DDR.

5 Implementation Details

In this chapter the implementation of the neural network developed in this thesis will be discussed. Furthermore, the transition from a conventional to a binarized network and details about the implementation in the SoC using OpenCL are presented.

5.1 Network Implementation

The Neural network is implemented as a multilayer network where all neurons of each layer are connected to all neurons of the next layer. Several implementations were tested that differantiate in the number of hidden layers.

5.1.1 Input Preprocessing

While data fed into the network can be any scale, a better accuracy can be achieved by scaling input data according to the activation functions point of operation. In the case of the EMNIST dataset we have 28x28 pixel of grayscale images. The values range from 0 to 255. This is resulting in 784 fields of input data. If the hyperbolic tangent function is chosen it is useful to scale the input values between -1 and 1. Also lots of different types of pre-filtering can be done in this step. The dynamics of the image could easily be changed by introducing a threshold from which pixels are stared being recognized.

5.1.2 Hidden Layers

The number of hidden layers in a neural network is usually growing with its complexity. One can image an additional layer of input neurons as an additional step of filtering attitudes out of the image. The network is implemented in a way that unlimited hidden layers can be implemented during initialization. Each hidden layer can have a distinct number of neurons.

While conventional neural networks usually work with a smaller amount of neurons inside the hidden layer, after initial testing it turned out useful to have at least 2048

neurons in each layer when using binarized neural networks. State-of-the-art results can be achieved starting with 2 layers, each consisting of 4096 neurons.

5.1.3 Activation Function

The activation function of the network reads the sum of all activations and is applying a mathematical function f(x) to it. This function can theoretically be any function imaginable. This network was tested with hyperbolic tangent, sigmoid, hardsigmoid and the step function.

In binarized networks, best results were achieved with the hyperbolic tangent function. Since hyperbolic tangent has a faster changing gradient around -1 and 1 as for example the sigmoid function does, during backpropagation the biggest changes of the weights can be achieved. This is needed in binarized networks, since they only support output values of -1 or 1 and need big canges in weights to make a switch from one output value to the other happen.

Since the derivate of the hyperbolic tangent shows bigger gradients on the points (1,1) and (-1,-1) as for example the sigmoid functions derivate, it also has bigger potential to learn binarized networks. This justifies by the fact that during training the back-propagation algorithm is used, where the error-signal is used as an input for the derivation of the activation function to determine the weights deltas. Binarized networks need big deltas in weights to change their output values from 1 to –1 or vice versa,

With conventional networks it is rather useful to stay inside the area around the zero-point of the activation function, since it allows intermediate instead of discrete values, small changes in the weights are needed to work best with the SGD algorithm.

```
1
2  float act_sigmoid(float x) {
3      return 1.0 / (1.0 + (exp(double)-x));
4  }
5
6  float der_sigmoid(float x) {
7      float s = act_sigmoid(x);
8      return s*(1.0-s);
9  }
10
11 float act_tanh(float x) {
12     return tanh(x);
13 }
14 float der_tanh(float x) {
15     return 1-pow(tanh(x),2);
16 }
17
18 float act_hard_sigmoid(float x) {
19     return max(0.0, min(1.0, (double)((x+1)/2)));
20 }
21 float act_hard_tanh(float x) {
22     return (2*act_hard_sigmoid(x))-1;
23 }
24 float act_step(float x) {
25     return x>=0?1:-1;
26 }
```

Listing 5.1: Implementation of the activation functions and their derivates

5.1.4 Weights Scaling

As described in the previous paragraph, binarized neural networks show the best performance around the points (1,1) and (-1,-1). To guarantee this it is required to directly hand the weighted inputs to the activation function after integrating them.

In an artificial neuron a sum over all weights is built. While with conventional neural networks it is useful to scale this sum by dividing it by the number of weights, in binarized networks it turned out to decrease the performance drastically. In conventional networks, linearity of an activation function is wanted, because it allows

for small in the output while only providing small changed to the input value. As introduced in chapter 3.1.4, during backpropagation a high gradient is needed

5.1.5 Forward Propagation

During forward propagation, a loop over all layers is performed, where in each iteration an OpenCL kernel is executed to calculate the dot product of the weights-matrix and the activation-vector. The implementations of all OpenCL Kernels implemented are described in detail in section 5.2. After calculating the dot product, the neurons activation function described in 5.1.3 is called.

5.1.6 Gradient Calculation

Courbariaux is implementing his work with help of the Theano framework, which enables gradient calculation using complex algorithms and thus resulting in a long training time. Since this works implementation is intended to run on embedded systems with limited resources, the choice was made to implement a simplified gradient calculation.

The cost $\Delta Cost$ of an output neuron is calculated by $Output{+}desired{-}Output_n$. The output layers neuron output is used as an input for the activation functions derivate and is then scaled by the factor $\Delta Cost * LearningRate$, resulting in $\Delta Weight$

5.1.7 Binarizing Weights

Courbariaux is implementing weights binarization in this Thenao implementation in a way that each node in the networks processing graph will binarize the weights prior to processing. This results in redundant binarization steps, calculating the same values for each node.

This work implements weight binarization in a way that prior to processing a ne matrix of weights is allocated, where the binary values are stored. As chapter 3.1.6.3 introduces, during training the original weights of floating point type, as well as the binarized weights are needed, to allocating the new binarized weight matrix is no redundant step. During each step in the training process, binarized and non-binarized weights are updated, if needed. This reduces the amount of binarization steps drastically and is resulting in a much faster training process.

The binarization itself is implemented by looping over all weights, while testing if the weight is greater or equal to zero.

5.1.8 Network Optimizations

During implementation several optimizations were applied to the binarized network to gain better results or lower the processing time. The following paragraphs will present and discuss those optimizations.

5.1.8.1 Parameter Initialization

The most simple form of initializing parameters would be to assign random parameters between -1 and 1 to all the weights. While this method is working will in networks with just 1 hidden layer, it has proven to

The networks weights are initialized with random numbers between -1 and 1. While this is showing a good learning performance on networks implementing 1 hidden layer, the performance decreases with the number of hidden layers. If the weights of a network are too small, passing multiple layers could make the signal until it reaches the output neurons. If they are too big, while passing though the network, they may grow too big and lose their usefulness.

In [51] Glorot is presenting a method of initializing the networks parameters in a reasonable range by making them depend on the amount of inputs and output connected to the layer. $Var(w) = \frac{2}{n_{in}+n_{out}}$

Glorot initialization, originally proposed by Xavier Glorot and Yoshua Bengio in [51], is an initialization technique for weights where the variance of a layers output neurons

While small networks, consisting only of a few layers, can be initialized by normal distribution with a mean of 0.0, for deep neural network with many layers other initialization methods allow a more efficient training.

Initializing deep networks with normal distribution and $\sigma = 0.01$ leads to the neurons activations very close to zero. By traversing through the network and applying for example the tanh activation for all neurons the activations will get smaller the closer they come to the output layer, which leads to the ”vanishing gradients” problem. During back propagation this leads to gradients close to zero in all layers which will result in inablity to learn properly.

This could be avoided by using $\sigma = 1.0$ for initialization with normal distribution. This can easily lead to saturation inside the activation function, so the neurons output will be either +1 or -1, which, again, makes gradient calculation tough, because without change in output values, the gradients will get close to zero, like they do with small variance.

Also, the exploding gradients problem can occur then weights and thus activations get too big.

$$weight_{scale} = 1/\sqrt{\frac{2}{n_i + n_{i+1}}} \tag{5.1}$$

Equation 5.1 describes how the weights are initialized.

5.1.8.2 Learning Rate Decay

The learning rate is one of the most important parameters to configure for neural networks.

It is responsible for scaling Δw and Δb in the direction of the gradient. Low training rates are in general more reliable because the steps taken for weight changes are very small and the neuron's error is recalculated with every step. The smaller these changes, the more steps are needed to reach towards the minimum of the loss function, so, the overall training time increases. If the learning rate is small and is reaching a local minimum of the error function, it might not be able to leave this minimum anymore, because the step size is too small [52]. While big changes can speed up the training process by bringing the parameters very fast close to the desired target, they may overshoot it and lead to a diverging behavior [53].

After each training cycle, the learning rate is multiplied by a static factor. Equation 5.2 is showing the calculation of the step size for one cycle, where n is the number of training cycles and η_{start} and η_{end} are the start- and end-point of the learning rate.

$$\Delta\eta = (\eta_{end}/\eta_{start})^{1.0/n} \tag{5.2}$$

Glorot Learning Rate Scaling

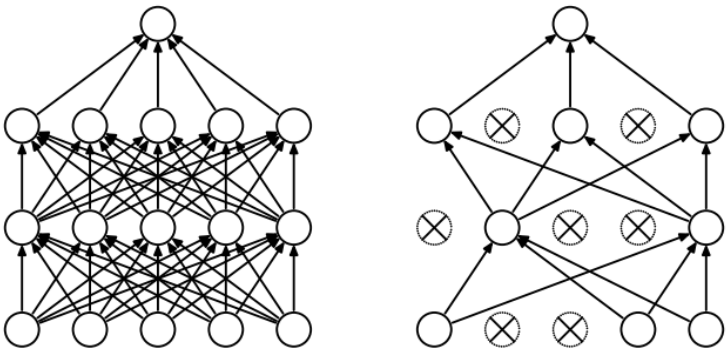
Analogous to the initialization of the networks weights presented in chapter 5.1.8.1, it is useful to scale weights initialization according to Glorots parameters [51]. To maintain those scaled value through the entire training process, also the learning rate can be scaled by the parameters Glorot defined. This process guarantees that the input to the activation function will stay in the range that Glorot proposes. The final scaling parameter is calculated by equation 5.3

$$lr_{scale} = 100/\sqrt{\frac{1.5}{n_i + n_{i-1}}} \tag{5.3}$$

In this implementation the factors slightly differ from Glorots proposal and the weight initialization chosen. During implementation other network optimizations were implemented and those parameters have statistically proven the best results.

5.1.8.3 Dropout

Dropout is a technique proposed in 2014 [54] that can overcome overfitting of large networks. It evolved from the technique of adding random noise to the networks parameters during the training process. Instead of adding noise, the network is thinned out by randomly dropping neurons together with their connections from the network, as figure 5.1 visualizes.



(a) Standard Neural Net (b) After applying dropout.

Figure 5.1: Visualization of the dropout technique. A depicting a default neural network. B depicts the same network thinned out by applying dropout. Crossed units are dropped. Image reused from [54].

5.1.8.4 Shuffle Dataset

For all methods implementing the SGD algorithms it is important for all training samples to be trained independently. While visiting the training samples in fixed order, as they are represented on the storage media, reduces random access to the disk, faster convergence of the networks approximation can be observed when accessing the training samples in random order. For a better convergance, the samples can be pre-shuffled on disk to increase convergance citeLeCun:backprop [55, pp. 123].

5.1.8.5 Exaggerated Targets

In the training process of neural networks, the cost for each output neurons error is calculated and backpropagated through the network to adjust the weights. While with conventional networks an error of 0 is almost never reached, because the output neurons activations are slowly converging towards the desired output, binarized networks do not show this behavior. Their output neurons activation function only has binary output values of [-1, 1] or [FALSE, TRUE], which leads to the fact that the error for some output neurons activation is 0, but the result still is wrong. The network paths leading to this result will not be touched during the learning process. Since in the training of binarized networks the with stochastic gradient descent algorithm is used, weights are, like in the training of conventional networks, adjusted by small changes to reach the global minimum of their error function.

For classification problems this can lead to problems: Most output activations are supposed to be FALSE, while only one output is supposed to be TRUE. In a situation where, during classification of numbers from 0 to 9, the wrong class is detected, only the path of 2 output neurons would be adjusted, while 8 are considered to have reached their global minimum. This can lead to long training times, because after the network made some initial learning progress, only few weights are adjusted at each training cycle.

In neuroscience the Hebbian theory [56] describes a basic mechanism for the adaptation of neurons in the brain during a learning process. Hebb describes:

> Let us assume that the persistence or repetition of a reverberatory activity (or "trace") tends to induce lasting cellular changes that add to its stability.[...] When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased.

This theory can be summarized as "What fires together wires together" and is a mechanism that can be observed often in nature: People tend to adapt behavioral pattern that work well in certain situations. A view on the same theory on a meta-scale. The theory can be interpreted and transferred to the problem in a way that connections between neurons should get stronger if they lead to the desired output.

Applying this theory to the problem described can be done by exaggerating the training data on the case of not detecting any error as shown in listing 5.2. By scaling the exaggeration with the learning rate, it can be adjusted through the learning process. During the end of the training, exsggeration will have less impact on the network, to avoid vanishing gradients.

```
float CostFunction(float input, float target)
{
    if(input == target) {
        float scale = scaleParameter * target;
        target += scale * learningRate;
    }
    return input - target;
}
```

Listing 5.2: Exaggerating the error of a neurons activation.

5.1.8.6 Converging Activation Function Replacement

To the user, the time needed for executing forward propagation is directly influencing the experience the user has with the network. The faster a network can classify an image, the more responsive it is.

One factor that can influence the forward propagation time is the type of activation function used in all neurons. On most hardware computational expensive functions like trigonometric functions are implemented with a combination of lockup tables and interpolation, which can lead to high computation times. Since this function is executed once for every neuron in the network, optimizations show a great influence on execution time.

As shown in chapter 3.1.5, the backpropagation algorithm requires differentiable functions to show best performance. Since easy to compute functions, like the step function or a saturated linear function are not differentiable or have a derivate of 1 for certain input ranges, they should be avoided to be used with the backpropagation algorithm.

An optimization to overcome the problematic, while still being able to make proper use of the backpropagation algorithms advantages is to use functions that converge around the desired input range. Examples of converging functions are presented in 5.2. While during backward propagation the computationally expensive function $f(x)$ is used, during forward propagation, it can be replaced by a simpler function $g(x)$ that converges to $f(x)$ in the desired input range.

This modification only shows good performance with binarized networks, because of the binarized output of the activation function, which allows a big input range to produce no change the functions resulting output value.

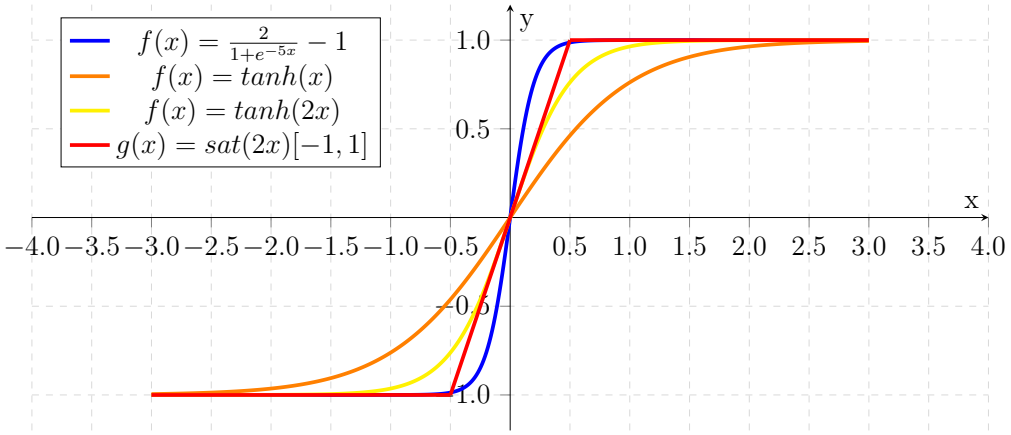


Figure 5.2: Converging functions that can replace each other.

5.2 OpenCL Implementations

To be able to record and compare measurements of different version of he neural network, several OpenCL kernels were created. All the implementations use a mixture of a NDRange kernel and a single-work-item kernel. The weights matrix consists of n rows and n_{-1} columns, where n is the amount of neurons of the layer that will be calculated and n_{-1} is the previous layers number of neurons.

Each row of a matrix calculation is implemented as a single-work-item kernel, meaning all the operations executed for processing this row are executed within one execution cycle of the kernel. Processing of multiple row is done by executing them inside a NDRange kernel. In NDRange kernels multiple threads of the same kernel-code are pipelined and executed by a thread scheduler.

Inside the single work item kernels all loop-carried dependencies were removed, by simply pre-caclulating them before the loop executes [57, p. 106]. All loops and nested loops have integer bound exit conditions to speed up execution and pointer aliasing is avoided by making proper use of the restrict keyword [57, p. 121].

5.2.1 Matrix Dot Product

Listing 5.3 is presenting an OpenCL kernel implementing a matrix dot product, which is used for processing inside conventional neural networks.

```
__kernel void vectorMult
  (__global const float * restrict weight,
   __global const float * restrict input_neuron,
   __global float * restrict output, int input_length)
{
  // get index of the work item
  // this equals the row we are processing
  int index = get_global_id(0);
  // row iterator
  int it_r = index * input_length;
  int sum = 0;
  // iterate through cols (it_r + i)
  // apply weight to input and sum up
  for(int i=0;i<input_length;i++)
  {
    sum += weight[it_r + i] * input_neuron[i]);
  }
  // write sum into output buffer
  output[index] = sum;
}
```

Listing 5.3: Implementation of a matrix dot product in OpenCL.

The kernel receives 2 input buffers for the neurons weights and the previous layers activation output, as well as an output buffer for this layers activation output. To allow optimization, the pointers to those buffers have the restrict type qualifier set. This indicates to the compiler that the memory the pointer points to in only accessed through this pointer. This is avoiding pointer aliasing and allows the OpenCL compiler to implement caching optimizations.

Multiple threads of this kernel are instantiated and scheduled by the OpenCL framework to process all the matrix's rows. An index is assigned to each thread, representing the rows index. This index can be read inside the kernel by receiving the global id from the OpenCL framework. This is done by calling the function `get_global_id(0)`.

The offset of the row to be processed in this instantiation of the kernel is calculated by the number of matrix columns multiplied by the amount of neurons inside the previous layer: `it_r = index * input_length;`

Then the sum of all weighted input is built by looping over all elements of the row that is processed and multiplying them with the last layers output of the corresponding

neuron: `sum += weight[it_r + i] * input_neuron[i];` After the sum is build, it is written back to the output buffer at the index of the thread.

5.2.2 XNOR Product

Listing 5.4 is showing the processing of the XNOR product within an OpenCL kernel. It is implemented analogous to the previously discussed dot product.

```
__kernel void vectorMult
  (__global const unsigned char * restrict weight,
   __global const unsigned char * restrict input_neuron,
   __global float * restrict output, int input_length)
{
  // get index of the work item
  // this equals the row we are processing
  int index = get_global_id(0);
  // row iterator
  int it_r = index * input_length;
  int sum = 0;
  // iterate through cols (it_r + i)
  // apply weight to input (xnor) and sum up
  for(int i=0;i<input_length;i++)
  {
    sum += (weight[it_r + i] == input_neuron[i])?1:-1;
  }
  // write sum into output buffer
  output[index] = sum;
}
```

Listing 5.4: Implementation of the XNOR product in OpenCL.

Since the XNOR product is used in binarized neural networks, the floating point input buffers can be replaced by buffers of the datatype `char`. For optimization purposes, the output buffer still has the datatype `float`. This is done, because the output values of the kernel still have to be used as the activation functions input parameters which are supposed to be floating point values.

The main difference to the kernel computing the dot product is how the weighted inputs are calculated: The multiplication from the dot product is replaced by the

XNOR logic function: `weights[it_r + i] == input_neuron[i];`. In C-style programming languages, the XNOR function can be represented by the equality operator `==`. As shown in table 3.1 the output of the XNOR function is only 1 if the input values are equal.

5.2.3 Compressed XNOR Product

```
__kernel void vectorMult
  (__global const unsigned int * restrict weight·,
   __global const unsigned int * restrict input_neuron·,
   __global int * restrict output, int input_length,
   int process_max)
{
  // get index of the work item
  // this equals the row we are processing
  int index = get_global_id(0);
  // row iterator
  int it_r = index * input_length;
  int sum = 0;
  int to_process = process_max;
  // iterate through cols (it_r + i)
  // apply weight to input and sum up
  for(int i=0;i<=input_length;i++)
  {
    for(int bit=0;bit<32;bit++)
    {
      if(to_process)
      {
        char a = (weight[it_r + i] >> bit) & 0x1;
        char b = (input_neuron[i] >> bit) & 0x1;
        sum += (a == b)?1:-1;
        to_process--;
      }
    }
  }
  // write sum into output buffer
  output[index] = sum;
}
```

Listing 5.5: Implementation of the compressed XNOR product in OpenCL.

For calculating the compressed XNOR product, as shown in listing 5.5, input values for weights and the output of last layers neurons are compressed by only using 1 bit for each values representation. 32 bits are stored into an unsigned integer.

To process the XNOR operation from the compressed buffers, the bits must be uncompressed by using the shift operation `(x >> bit) & 0x01`. Since it can not be guaranteed that the amount of weights or inputs are multiples of 32, the exact amount must be set as the input parameter `process_max`.

Usually the XNOR operation could be implemented as a bitwise operation over the whole 32 bit integer by negating a XOR operation: `xnor = ~(A^B)`. It was chosen not to use this implementation, because still the sum of the results of the XNOR operation has to be built by looping over all bits. Doing the bitwise operation while still implementing the loop would use more resources.

6 Analysis

In this chapter performance measurements of practical outcome of this thesis will be presented. They will be discussed according to the performance indicators defined in this thesis and compared to measurements of conventional neural networks as well as to other authors results.

6.1 Performance Measurements

The following section will present measurements of implementations of a neural network predicting the EMNIST classification benchmark

The implemented network consists of 1 input layer 1 output layer and 1 or 2 hidden layers with 4096 neurons each. Measurements on processing times of the single layers are taken and presented.

6.1.1 Classification Accuracy

The classification accuracy is the percentage of images classified correctly. Figure 6.1 lists the accuracy for different implementations of MNIST handwritten number classification. Compared are four implementations derived from this thesis and two implementations implemented by other authors.

Implementation	**Classification Accuracy**
1 Hidden Layer ANN	97%
1 Hidden Layer XNOR	87%
2 Hidden Layer ANN	98%
2 Hidden Layer XNOR	92%
Huynh	97%
Courbariaux	96%

Table 6.1: Classification Accuracy for different network models.

The networks implemented in this paper are 1 and 2 hidden layer implementations of the XNOR network and a conventional network with its operating point close to (0,0). While the conventional implementation gives an accuracy 0f 97-98%, the BNN implementation gives accuracy of 87-92%. Huynh states and accuracy of 97% [5], Courbariaux 96% [4].

6.1.2 Execution-Time Per Frame

Besides the accuracy of the network, the execution time is a good measurement for the networks performance. The following tables show processing times for ARM-only implementations compared to FPGA-accelerated implementations of the same network. The measurements only contain the kernels processing time, excluding the time needed to transfer the images from ARM to FPGA core. This is done, because in a real-life scenario images are usually coming from an outside source, like a camera. Since the FPGA core has direct access to all the SoCs peripherals, the measurements of interest are the raw kernel processing times.

ARM Implementation

Table 6.2 is showing the processing time for the layers involved in performing the classification. The column **ANN** lists timings for the conventional artificial neural network implementation, using floating-point multiplication for weights and activations. Column **XNOR** shows timings for the XNOR implementation of the network using 8bit integers to store weights and activations. The column **Compressed** lists timings for the compressed XNOR network, where weights and activations are stored as 1-bit values.

Resource	**ANN**	**XNOR**	**Compressed**
Hidden Layer 1 (784x4096)	308.066 ms	103.102 ms	144.344 ms
Hidden Layer 2 (4096x4096)	1654.475 ms	687.858 ms	963.014 ms
Output Layer (10x4096)	4.047 ms	1.683 ms	2.641 ms
1 Hidden Layer Network	312.145 ms	131.783 ms	147.028 ms
2 Hidden Layer Network	1966.782 ms	819.191 ms	1110.042 ms

Table 6.2: Timings for ARM-based implementation of a conventional artificial neural network.

SoC ANN Implementation

Table 6.3 lists processing times using the ARM core and accelerating matrix operations with the FPGA. The column **ANN** lists timings for the conventional artificial neural network implementation, using floating-point multiplication for weights and activations. Column **XNOR** shows timings for the XNOR implementation of the network using 8bit integers to store weights and activations. The column **Compressed** lists timings for the compressed XNOR network, where weights and activations are stored as 1-bit values.

Resource	ANN	XNOR	Compressed
Hidden Layer 1 (784x4096)	45.812 ms	45.315 ms	32.966 ms
Hidden Layer 2 (4096x4096)	458.964 ms	439.686 ms	205.096 ms
Output Layer (10x4096)	2.529 ms	1.802 ms	1.612 ms
1 Hidden Layer Network	48.341 ms	46.952 ms	34.578 ms
2 Hidden Layer Network	507.305 ms	486.803 ms	239.674 ms

Table 6.3: Timings for SoC-based implementation of a conventional artificial neural network.

6.1.3 FPGA Logic Gates used

Table 6.4 lists the amount of FPGA resources used in different implementations of image classification networks running on FPGA hardware. Listed are other authors implementations compared to 3 different implementations derived from this work: An artificial neural network using floating point weights and activations and implementing floating point calculation, the XNOR implementation using 8bit weights and activations with the values -1 or 1 and the compressed version, using 1-bit representation of weights and activations. A detailed usage of resources from these implementations is listed in table 6.5.

Resource	ALUTs	Registers	Logic	DSP Blocks	Memory Blocks
Multiplication	8,992	10,679	38%	3	62
XNOR	8,316	10,236	37%	2	60
Compressed	8,570	10,470	38%	2	63
Huynh	63,454	44,080	92%	64	N/A
Park-Sung	121,173	130,802	N/A	900	232
wang-et-al	N/A	16,356	84%	93	65

Table 6.4: FPGA Resources allocated compared between different implementations.

Park and Sun [6] implement a network using only 3 bits for storing the networks weights. The network consists of 784-1022-1022-1022-10 neurons, so 3 hidden layers are implemented. 100 images are preloaded in the FPGAs internal memory and are iterated during runtime.

Wang et al. [7] implement a discrete model in FPA hardware only. The images are preprocessed by de-noising, edge amplifying, binarization and removing redundant information. Afterwards the images are fed into a network consisting of 784-500-500-2000-10 neurons, so 3 hidden layers are implemented. The image classified is preloaded into the FPGA's internal memory and there is no mechanism for changing the picture during runtime.

Huynh [5] implements different version of a network, ranging from 784-40-10 to 784-126-126-126-10. For this measurement an implementation of 784-40-40-40-10 is considered. The network uses 16bit floating point weights and activations.

Resource	**Multiplication**	**XNOR**	**Compressed**	max
ALUTs	8,992	8,316	8,570	N/A
Registers	10,679	10,236	10,470	N/A
Logic	38%	37%	38%	15,880
I/O Pins	103	103	103	314
DSP BLOCKS	3	2	2	84
Memory Bits	347,360	346,240	362,880	2,764,800
M10K blocks	62	60	63	270
max fanout	615	606	609	N/A

Table 6.5: Resources allocated in FPGA for different kernels

6.1.4 Training Cost

Table 6.6 is presenting the average training times for different network implementations. Compared are the implementation described in [1], implemented in the Theano framework using the Python programming language and the implementations developed in this thesis, all implemented in C++.

The training was performed on an Intel(R) Core(TM) i5-2520M CPU clocked at 2.50GHz. The measurements were taken after performing 60000 training cycles on 28x28 pixel grey-scale images.

e

Resource	1h layer training	2h layer training
Courbariaux	9682 min	21547 min
Multiply	1263 min	8154 min
XNOR	242 min	843 min
Compressed	256 min	867 min

Table 6.6: Training Times for different network implementations.

6.1.5 Development Cost

The development cost of a software project can be measured in lines-of-code (LOC) the finished product implements [50, pp. 55-56]. Table 6.7 is listing the average ammount of lines-of-code derived from several GitHub projects implementing the MNIST dataset benchmark and from the project described in this thesis.

Resource	Lines-Of-Code
VHDL	25470
Verilog	12684
OpenCL(ARM+FPGA	2026
OpenCL(FPGA-only)	47

Table 6.7: Training Times for different network implementations.

6.2 Evaluation of KPI

In this section measurements of key performance indicators listed in the previous section are evaluated.

6.2.1 Classification Accuracy

As seen in figure 6.1, all models under investigation show state of the art results $> 85\%$ classification accuracy. While conventional neural networks already show results $> 90\%$ with just 1 hidden layer with only, binarized neural networks drastically improve by involving a second hidden layer, both containing at least 2048 neurons. This can be explained by the fact that conventional networks implement the activation function being able to output all possible values between -1 and 1. Like this it is quite easy to identify a trend the output will follow: The output value will shift more and more towards the correct value and already by seeing a result of 0.2 instead of 1.0, a trend of the outcoming value can already been foreseen and a lower

threshold can be applied. With a lowered threshold, the training time can drastically be shortened compared to training models of the XNOR network.

Compared with other authors implementations, the XNOR implementation presented in this thesis is showing slightly less classification accuracy. While other authors implementations require a training time ranging from several days up to weeks, this model is trained in less than 1 day. Additionally, optimizations implemented in other authors implementations are not implemented in this thesis's work. Those include batch normalization and complex gradient calculation.

6.2.2 Execution-Time per Frame

The execution-time per frame is drastically lowered by accelerating the network with the FPGA core compared to an ARM-only implementation. Due to the FPGAs capability of parallelizing tasks, all weights of a neuron can be processed at once, while on ARM all calculations are executed serialized.

While the XNOR implementation is showing a high accelerating potential over the floating-point multiplication on an ARM core implementation, it has almost no impact using OpenCL running on FPGA fabric. Modern FPGA systems contain DSP blocks that implement native floating-point accelerators [58]. Using floating-point accelerators a floating-point calculation can be processed within 1 clock cycle which is the same timing a XNOR operation stored in a flipflop has. By compressing the network to only use 1-bit per weight and activation, even more operations can be parallelized. Since OpenCL uses C-style datatypes, 32 weights are stored into a 32-bit integer, instead of 1 weight per integer.

Thus, the biggest difference can be achieved when accelerating conventional artificial networks with OpenCL running on FPGA fabric.

Resource	**784x4096**	**4096x4096**	**10x4096**	**1h - 4096**	**2h - 4096**
ARM-mult	308.066 ms	1654.475 ms	4.047 ms	312.145 ms	1966.782 ms
ARM-xnor	103.102 ms	687.858 ms	1.683 ms	131.783 ms	819.191 ms
ARM-comp	144.244 ms	963.014 ms	2.641 ms	147.028 ms	1110.042 ms
SoC-mult	45.812 ms	458.964 ms	2.529 ms	48.952 ms	507.305 ms
SoC-xnor	45.315 ms	439.686 ms	1.802 ms	46.952 ms	486.803 ms
SoC-comp	32.966 ms	205.096 ms	1.612 ms	34.578 ms	239.674 ms

Table 6.8: Speed for matrix operations with different kernels

As table 6.8 shows, the processing time on ARM architecture is growing linear with the networks size. It takes 7.8 times longer to process a kernel with 784 neurons, as it

takes processing one with 10 neurons. Using the FPGA architecture, it is possible to parallelize processing, leading to non-linear growth of processing time over network sizes.

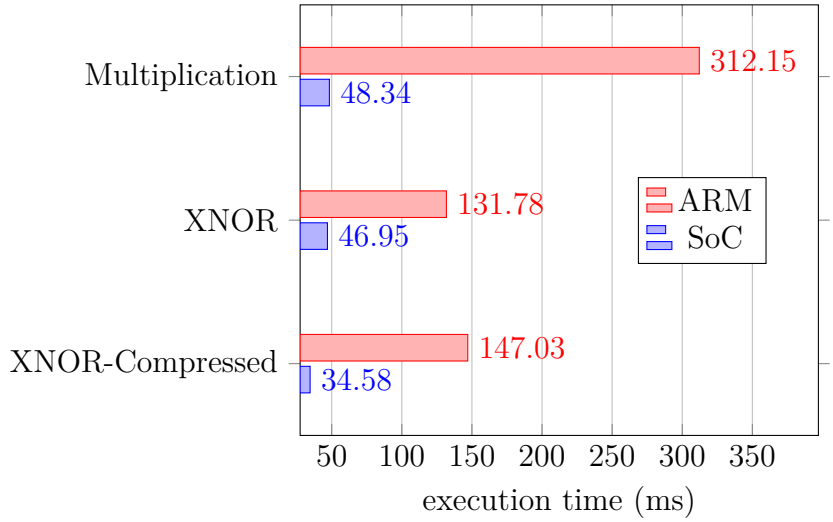


Figure 6.1: SoC kernel timings in comparision to ARM kernel timings for 1 hidden layer network.



Figure 6.2: SoC kernel timings in comparision to ARM kernel timings for 2 hidden layers network.

6.2.3 FPGA Logic Gates used

As shown in figure 6.3, using OpenCL to implement the compressed XNOR implementation is drastically saving FPGA resources compared to all other authors implementations. This is mainly due to the fact that the algorithm is only partly executed inside the FPGA. The FPGA does not take care of loading images or iterating through single layers. Only the matrix multiplication is accelerated inside the FPGA which is implemented in a way that the same implementation can be reused for all sizes. A downside of this implementation is that for small kernel sizes, which usually cold be parallelized to execute in just 1 clock cycle, the execution time is slightly higher as expected and thus saving almost no time compared to the ARM implementation.

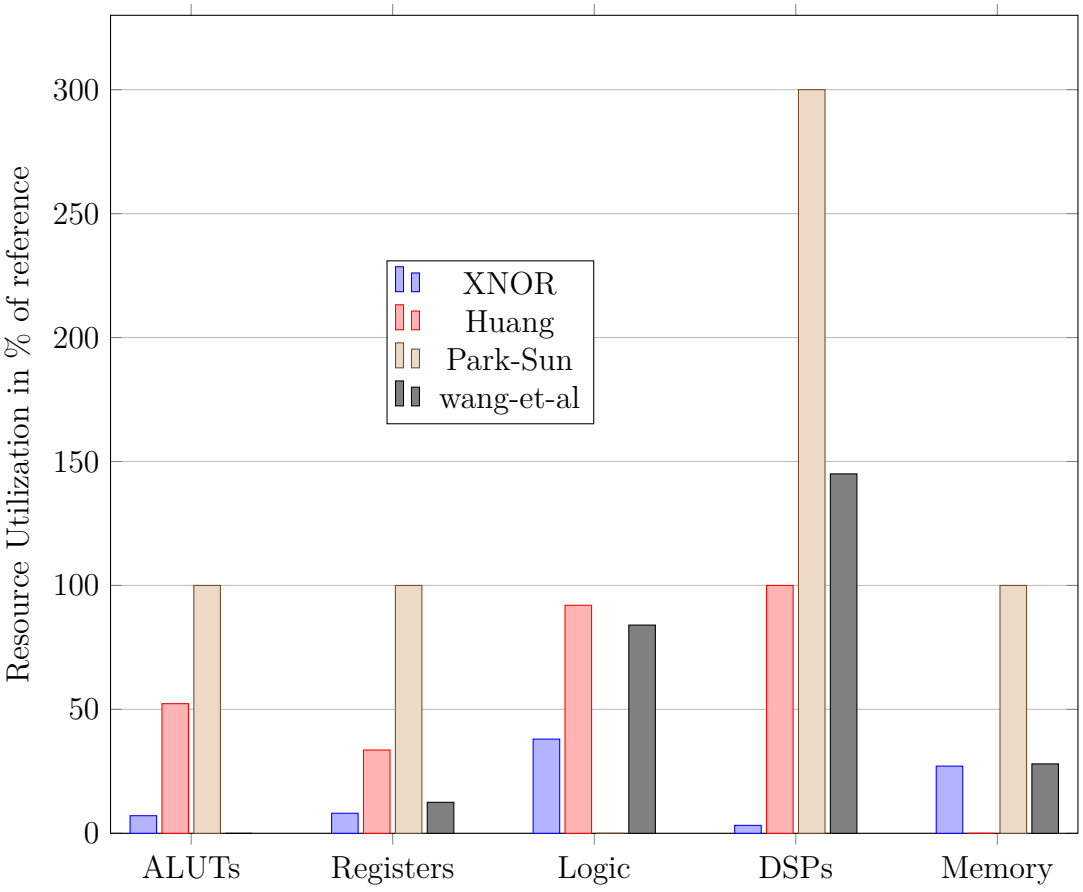


Figure 6.3: Comparison of logic utilization for different implementations.

6.2.4 Training Cost

As shown in Table 6.6, a native implementation in C++ is drastically deceasing the training time needed for learning a dataset of 60000 images, compared to an implementation using the Python programming language and the Theano framework. While Courbariaux's implementation needs up to 2 weeks for learning the dataset, the same operation can be achieved in 14 hours using a native C++ implementation.

This can mostly be explained by the way the Theano framework is performing network operations. Theano is internally building a network graph where all operations are an entity fully able to work on its own. The graphs nodes are executed serialized, where for each operation the network is binarized before and de-binarized after the operation. Also, the implementation includes additional steps to optimize the learning, like building a mean over a matrix of weights. This can be visualized by having each node in the graph performing a full loop over the networks neurons, executing just one operation. Together with the (de-)binarization, for each node in the graph there are 3 loops over all networks neurons.

The native C++ implementation is not implemented as a graph, and thus can make use of optimizations like loop unrolling, where multiple operations can be performed in just one loop over the networks neurons.

Also, the usage of the programming language itself is an explanation for the low training-time: Python is an interpreted dynamic language, meaning in its basic form, each line of code is interpreted and executed during runtime, while in C++ the compiler can produce highly optimized code during compile time. The implementation of basic features in the language differ a lot. A loop in Python for example is, when using the *range* implementation, very different from the for loop in C++: First, all elements described in the *range* function are pre-calculated to build an input vector. This vector is then used in a for-each-style loop to execute the operations on it's elements. In C++ at the end of each loop the next elements in calculated, which eliminates the pre-calculation of the input vector. When using a big range of inputs, like it is usually done in neural networks, this pre-caclulation can have a big impact on execution time. Iterating through a simple hidden layer of 4096 neurons is basically resulting in $4096 * 4096 = 16777216$ operations of multiplying weights with inputs.

6.2.5 Development Cost

The difference in lines-of-code, listed in Table 6.7, can be explained by the choice of programming language and the level of hardware abstraction OpenCL offers. As Table 6.9 shows, the intermediate VHDL and Verilog code, the OpenCL compiler

generates for this thesis's project has 581494 lines of code. Compared to the 47 lines of code needed for writing the FPGA kernel itself, this is scaled by a factor of approximately 10000. The resulting VHDL or Verilog code contains functionalities for transferring memory from the ARM cores to the FPGA fabric and back and brings state machines for executing the main software's threads, as well as helper threads, executing the data transfer or pre-calculating resources.

Language	**Lines-Of-Code**
VHDL	430634
Verilog	150860
Sum	581494

Table 6.9: Resulting lines-of-code in the intermediate code OpenCL generates.

Besides just comparing the lines of code needed for solving the problem, the development cost also includes the complexity of the act of writing the code. Writing VHDL and verilog code forces the developer to bring hardware architectural knowledge, since all operations have to be implemented on a register level. In comparison writing OpenCL code let the developer focus on solving the problem or algorithm on a higher abstraction level, without taking care of FPGA resources. So not only the time needed to write 1 lines of code is considered longer for writing VHDL code, also the salary of a VHDL developer is usually higher, so overall the usage of OpenCL is drastically lowering the development costs.

7 Conclusions

This chapter concludes the thesis with a discussion of its findings and contributions. Limitations, as well as outlines will be pointed out and directions for future research will be discussed.

7.1 Summary

7.2 Future Work

This work could prove the feasibility of implementing binarized neural networks using OpenCL on FPGA fabric.

Outsourcing other steps performed within the backpropagation algorithm could improve training times drastically. Steps within the backpropagation algorithm are very similar to the dot product accelerated in this work. They are implemented as multiplications of matrices and vectors. and thus, could easily be ported to be executed within the FPGA core.

This thesis did not examine the problem of memory bandwidth between ARM core and FPGA. This should be addressed by further research.

While this work is using dynamic kernels that can be reused with any network size, the algorithm could be even further parallelized by implementing static kernels for specific kernel sizes. Multiple kernels implementing memory pipelines could be used to transfer ones kernels result to the next kernels which can use them as input parameters. This would even lower processing times further and would avoid unneeded memory transfers between ARM- and FPGA core.

The next iteration step should implement convolutional neural networks using this technique, which would enable classification of sub-images inside a big image that can be classified in any angle. Also, the image source should not be a static picture that is fed from ARM core to the FPGA, but rather be a live image from a camera source, directly fetched by the FPGA. By training the network with the CIFAR-100 dataset, classes of real-life objects can be classified instead of handwritten objects.

Implementing multiple kernels in parallel would enable a real-time classification of all detectable objects within a camera stream.

Attaching the FPGA driven convolutional network to a cloud service, images of unidentified object can be automatically sent to the cloud provider and be identified by other ways, like humans. These new datasets can then be used to train an updated model of the network which is sent back to the processing system. Like this the network could easily adapt to changes in the environment.

Bibliography

[1] M. Courbariaux and Y. Bengio, "Binarynet: Training deep neural networks with weights and activations constrained to +1 or -1," vol. abs/1602.02830, 2016. [Online]. Available: http://arxiv.org/abs/1602.02830

[2] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," *CoRR*, vol. abs/1603.05279, 2016. [Online]. Available: http://arxiv.org/abs/1603.05279

[3] M. Kim and P. Smaragdis, "Bitwise neural networks," *CoRR*, vol. abs/1601.06071, 2016.

[4] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: an extension of MNIST to handwritten letters," vol. abs/1702.05373, 2017. [Online]. Available: http://arxiv.org/abs/1702.05373

[5] T. V. Huynh, "Deep neural network accelerator based on FPGA," in *2017 4th NAFOSTED Conference on Information and Computer Science*, Nov 2017, pp. 254–257.

[6] J. Park and W. Sung, "FPGA based implementation of deep neural networks using on-chip memory only," in *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, March 2016, pp. 1011–1015.

[7] Wang, Yang, Xu, and Fu, "Design of FPGA-based handwriting image recognition system," in *International Information and Engineering Technology Association, Advances In Modelling and Analysis B*, vol. 60, no. 2, 2017, pp. 493–504.

[8] A. Sehgal and N. Kehtarnavaz, "A convolutional neural network smartphone app for real-time voice activity detection," *IEEE Access*, vol. 6, pp. 9017–9026, 2018.

[9] L. Zhang, X. Wu, and D. Luo, "Real-time activity recognition on smartphones using deep neural networks," in *2015 IEEE 12th Intl Conf on Ubiquitous Intelligence and Computing and 2015 IEEE 12th Intl Conf on Autonomic and Trusted Computing and 2015 IEEE 15th Intl Conf on Scalable Computing and Communications and Its Associated Workshops (UIC-ATC-ScalCom)*, Aug 2015, pp. 1236–1242.

[10] L. Deng, "The mnist database of handwritten digit images for machine learning research [best of the web]," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, Nov 2012.

[11] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. S. Bernstein, A. C. Berg, and F. Li, "Imagenet large scale visual recognition challenge," *CoRR*, vol. abs/1409.0575, 2014.

[12] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vision*, vol. 60, no. 2, pp. 91–110, Nov. 2004. [Online]. Available: https://doi.org/10.1023/B:VISI.0000029664.99615.94

[13] R. T. Ionescu and M. Popescu, *Knowledge Transfer between Computer Vision and Text Mining*, 01 2016.

[14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems 25*, F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2012, pp. 1097–1105. [Online]. Available: http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf

[15] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *CoRR*, vol. abs/1409.1556, 2014.

[16] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. E. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," *CoRR*, vol. abs/1409.4842, 2014.

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015.

[18] L. A. Gatys, A. S. Ecker, and M. Bethge, "A neural algorithm of artistic style," *CoRR*, vol. abs/1508.06576, 2015. [Online]. Available: http://arxiv.org/abs/1508.06576

[19] E. Schikuta and E. Mann, "N2sky - neural networks as services in the clouds," *CoRR*, vol. abs/1401.2468, 2014. [Online]. Available: http://arxiv.org/abs/1401.2468

[20] S. Anwar, K. Hwang, and W. Sung, "Structured pruning of deep convolutional neural networks," *J. Emerg. Technol. Comput. Syst.*, vol. 13, no. 3, pp. 32:1–32:18, Feb. 2017. [Online]. Available: http://doi.acm.org/10.1145/3005348

[21] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient transfer learning," *CoRR*, vol. abs/1611.06440, 2016.

[22] Y. L. Cun, J. S. Denker, and S. A. Solla, "Advances in neural information processing systems 2," D. S. Touretzky, Ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1990, ch. Optimal Brain Damage, pp. 598–605. [Online]. Available: http://dl.acm.org/citation.cfm?id=109230.109298

[23] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *CoRR*, vol. abs/1704.04861, 2017.

[24] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," *CoRR*, vol. abs/1610.02357, 2016.

[25] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," *CoRR*, vol. abs/1707.01083, 2017.

[26] M. Courbariaux, Y. Bengio, and J. David, "Low precision arithmetic for deep learning," *CoRR*, vol. abs/1412.7024, 2014. [Online]. Available: http://arxiv.org/abs/1412.7024

[27] K. Hazelwood, S. Bird, D. Brooks, S. Chintala, U. Diril, D. Dzhulgakov, M. Fawzy, B. Jia, Y. Jia, A. Kalro, J. Law, K. Lee, J. Lu, P. Noordhuis, M. Smelyanskiy, L. Xiong, and X. Wang, "Applied machine learning at facebook: A datacenter infrastructure perspective," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2018, pp. 620–629.

[28] N. P. Jouppi, C. Young, N. Patil, D. A. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, R. C. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snelham, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-datacenter performance analysis of a tensor processing unit," *CoRR*, vol. abs/1704.04760, 2017. [Online]. Available: https://arxiv.org/abs/1704.04760

[29] A. M. Caulfield, E. S. Chung, A. Putnam, H. Angepat, J. Fowers, M. Haselman, S. Heil, M. Humphrey, P. Kaur, J. Y. Kim, D. Lo, T. Massengill, K. Ovtcharov, M. Papamichael, L. Woods, S. Lanka, D. Chiou, and D. Burger, "A cloud-scale acceleration architecture," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct 2016, pp. 1–13.

[30] P. Sundararajan and C. Kulkarni, "Data store acceleration-as-a-service on amazon fpga instances," 2017. [Online]. Available: https://www.xilinx.com/support/documentation/product-briefs/reniac-aws-f1.pdf

[31] A. Putnam, A. Caulfield, E. Chung, D. Chiou, K. Constantinides, J. Demme, H. Esmaeilzadeh, J. Fowers, J. Gray, M. Haselman, S. Hauck, S. Heil, A. Hormati, J.-Y. Kim, S. Lanka, E. Peterson, A. Smith, J. Thong, P. Y. Xiao, D. Burger, J. Larus, G. P. Gopal, and S. Pope, "A reconfigurable fabric for accelerating large-scale datacenter services," in *Proceeding of the 41st Annual International Symposium on Computer Architecuture (ISCA)*. IEEE Press, June 2014, pp. 13–24. [Online]. Available: https://www.microsoft.com/en-us/research/publication/a-reconfigurable-fabric-for-accelerating-large-scale-datacenter-services/

[32] F. Rosenblatt, *Principles of neurodynamics: perceptrons and the theory of brain mechanisms*, ser. Report (Cornell Aeronautical Laboratory). Spartan Books, 1962. [Online]. Available: https://books.google.de/books?id=7FhRAAAAMAAJ

[33] R. Rojas, *Neural Networks: A Systematic Introduction*. Berlin, Heidelberg: Springer-Verlag, 1996.

[34] J. Raju, S. Kumar, and L. Sneha, "Realization of logic gates using mcculloch-pitts neuron model," vol. 45, pp. 52–56, 03 2017.

[35] X. Li and X. Wu, "Constructing long short-term memory based deep recurrent neural networks for large vocabulary speech recognition," *CoRR*, vol. abs/1410.4281, 2014.

[36] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proceedings of the 30th International Conference on International Conference on Machine Learning - Volume 28*, ser. ICML'13. JMLR.org, 2013, pp. III–1310–III–1318. [Online]. Available: http://dl.acm.org/citation.cfm?id=3042817.3043083

[37] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Netw.*, vol. 4, no. 2, pp. 251–257, Mar. 1991. [Online]. Available: http://dx.doi.org/10.1016/0893-6080(91)90009-T

[38] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, http://www.deeplearningbook.org.

[39] T. Masters, *Practical Neural Network Recipes in C++*. San Diego, CA, USA: Academic Press Professional, Inc., 1993.

[40] G. E. Hinton, S. Osindero, and Y.-W. Teh, "A fast learning algorithm for deep belief nets," *Neural Comput.*, vol. 18, no. 7, pp. 1527–1554, jul 2006. [Online]. Available: http://dx.doi.org/10.1162/neco.2006.18.7.1527

[41] K. Fukushima and S. Miyake, "Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition," in *Competition and Cooperation in Neural Nets*, S.-i. Amari and M. A. Arbib, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1982, pp. 267–285.

[42] W. Zhang, K. Itoh, J. Tanida, and Y. Ichioka, "Parallel distributed processing model with local space-invariant interconnections and its optical architecture," *Appl. Opt.*, vol. 29, no. 32, pp. 4790–4797, Nov 1990. [Online]. Available: http://ao.osa.org/abstract.cfm?URI=ao-29-32-4790

[43] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov 1998.

[44] S. Albelwi and A. Mahmood, "A framework for designing the architectures of deep convolutional neural networks," *Entropy*, vol. 19, no. 6, 2017. [Online]. Available: http://www.mdpi.com/1099-4300/19/6/242

[45] L. Bottou and O. Bousquet, "The tradeoffs of large scale learning," in *Advances in Neural Information Processing Systems*, J. Platt, D. Koller, Y. Singer, and S. Roweis, Eds. NIPS Foundation (http://books.nips.cc), 2008, vol. 20, pp. 161–168. [Online]. Available: http://leon.bottou.org/papers/bottou-bousquet-2008

[46] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient backprop," in *Neural Networks: Tricks of the Trade, This Book is an Outgrowth of a 1996 NIPS Workshop*. London, UK, UK: Springer-Verlag, 1998, pp. 9–50. [Online]. Available: http://dl.acm.org/citation.cfm?id=645754.668382

[47] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Neurocomputing: Foundations of research," J. A. Anderson and E. Rosenfeld, Eds. Cambridge, MA, USA: MIT Press, 1988, ch. Learning Representations by Back-propagating Errors, pp. 696–699. [Online]. Available: http://dl.acm.org/citation.cfm?id=65669.104451

[48] M. Courbariaux, Y. Bengio, and J. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," *CoRR*, vol. abs/1511.00363, 2015. [Online]. Available: http://arxiv.org/abs/1511.00363

[49] *OpenCL on FPGAs for GPU Programmers*, Accelware, 6 2014, version 1.0.

[50] S. McConnell, *Software Estimation - Demystifying the Black Art.* Microsoft Press, 2006.

[51] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *In Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS'10). Society for Artificial Intelligence and Statistics*, 2010.

[52] Y. Bengio, N. Boulanger-Lewandowski, and R. Pascanu, "Advances in optimizing recurrent networks," *CoRR*, vol. abs/1212.0901, 2012. [Online]. Available: http://arxiv.org/abs/1212.0901

[53] L. N. Smith, "Cyclical learning rates for training neural networks," in *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, March 2017, pp. 464–472.

[54] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. [Online]. Available: http://jmlr.org/papers/v15/srivastava14a.html

[55] Y. Bengio, "Practical recommendations for gradient-based training of deep architectures," *CoRR*, vol. abs/1206.5533, 2012. [Online]. Available: http://arxiv.org/abs/1206.5533

[56] D. Hebb, *The Organization of Behavior: A Neuropsychological Theory.* Taylor & Francis, 1949.

[57] *Intel FPGA SDK for OpenCL Pro Edition - Best Practices Guide*, Intel.

[58] *Cyclone V Device Datasheet*, Intel, 5 2018.